ABAP Development Tools for Eclipse

Generated on: 2026-04-04 19:23:31 GMT+0000

ABAP Cloud | ABAP Cross-Product

Public

Original content: https://help.sap.com/docs/ABAP_Cloud/bbcee501b99848bdadecd4e290db3ae4?locale=en-US&state=PRODUCTION&version=sap_cross_product_abap

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit https://help.sap.com/docs/disclaimer.

SAP Joule for Developers, ABAP AI Capabilities

SAP Joule for developers, ABAP AI capabilities supports you in the end-to-end development using generative AI. It can be integrated into ABAP development tools for Eclipse.

Note

To leverage Joule's capabilities, you need to purchase an additional license: For SAP BTP ABAP environment and SAP S/4HANA Cloud Public Edition, see SAP Note 3571857.

For more information on how to integrate SAP Joule for developers, ABAP AI capabilities into ABAP development tools for Eclipse, see Getting Started.

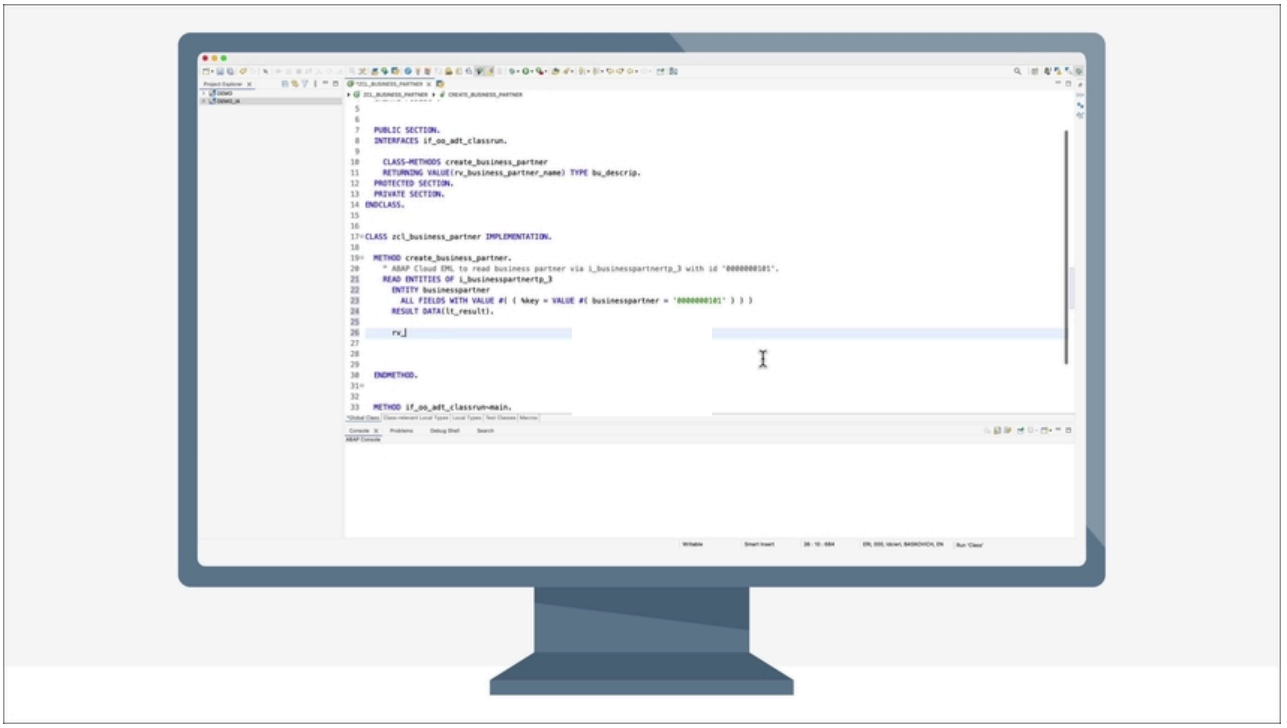
SAP Joule for developers, ABAP AI capabilities is designed to support you with your day-to-day tasks when working with ABAP code, to increase your efficiency and to lower total cost of development.

Related Information

Getting AI Coding Assistance

Getting AI Coding Assistance

Get a first impression of SAP Joule for developers, ABAP AI capabilities, explore its capabilities, and learn more about its advantages.



Open this video in a new window

Related Information

Learn

Recommendations and Constraints

Start

The content in this *Start* section provides you with the essentials to get started with SAP Joule for developers, ABAP AI capabilities immediately.

Getting Started

Learn more how you can get SAP Joule for developers, ABAP AI capabilities and which business roles are required.

Quick Start

If you're eager to get started, here's a short instruction that shows you how to access SAP Joule for developers, ABAP AI capabilities to try out its capabilities.

Getting Started

Learn more how you can get SAP Joule for developers, ABAP AI capabilities and which business roles are required.

Roles and Authorizations

To be able to use SAP Joule for developers, ABAP AI capabilities, the following prerequisites need to be met:

- Your system administrator must assign you to a business role that contains the business catalog `SAP_A4C_BC_DEV_AIQ_PC`.
- Your system administrator must assign the values AIQ_TYPE = [JADC_CODE,JADC_TEXT] and ACTVT = AF for the authorization object `S_AIQADTLO`.
- Additionally, the latest version of ABAP development tools for Eclipse is required.

Note

To leverage Joule's capabilities, you need to purchase an additional license: For SAP BTP ABAP environment and SAP S/4HANA Cloud Public Edition, see SAP Note 3571857.

Subscribing to SAP Joule for Developers, ABAP AI Capabilities

Here's how you as an administrator can subscribe to SAP Joule for developers, ABAP AI capabilities.

Prerequisites

Note

Please be aware that this procedure is only relevant if you're working in SAP S/4HANA Cloud Private Edition.

Please ensure that the following requirements are met before subscribing to SAP Joule for developers, ABAP AI capabilities:

- You have a subaccount in SAP BTP, Cloud Foundry environment. See Managing Subaccounts Using the Cockpit.

Note

When creating your subaccount, choose a region that is geographically close to you to optimize the performance of SAP Joule for developers, ABAP AI capabilities.

- You have activated the SAP Joule for Developers package in SAP for Me. See Business AI (SKU8019164) Activation and User Assignment Enablement Guide.
- You have established trust to the IAS tenant in the SAP BTP Cockpit. See Establish Trust and Federation of Custom Identity Providers for Platform Users.
- You have the necessary `j4d-abap` entitlement in your subaccount. See Configure Entitlements and Quotas for Subaccounts.

You can now follow the below procedure to subscribe to SAP Joule for developers, ABAP AI capabilities.

ℹ Note

Working With "EU Only" Access – If you want to work in an "EU only" mode, an EU only region has to be selected when subscribing to the AI Hub (the subscription must be done by your admin).

Procedure

1. Log into the SAP BTP Cockpit and navigate to your subaccount.
2. Under the **Services** menu, select **Instances and Subscriptions**.
3. Click on the **Create** button and select **SAP Joule for developers, ABAP AI capabilities** as the service.
4. Under the **Plan** dropdown, select **Default**. If this option is not available, you may need to adjust the entitlements for your subaccount.
5. Click on the **Create** button to finalize the subscription.

Results

You have successfully subscribed to SAP Joule for developers, ABAP AI capabilities.

ℹ Note

If you need support or experience issues, please report an incident under component `BC-AI-GEN-HUB`.

Configuring Your ABAP System

As an administrator, you wish to enable SAP Joule for developers, ABAP AI capabilities to get AI-generated code completion suggestions. Follow the steps below to configure you ABAP system.

ℹ Note

Please be aware that this procedure is only relevant if you're working in SAP S/4HANA Cloud Private Edition.

Prerequisites

- You are administrator in the ABAP system where you wish to enable SAP Joule for developers, ABAP AI capabilities.
- You've subscribed to SAP Joule for developers, ABAP AI capabilities. See Subscribing to SAP Joule for Developers, ABAP AI Capabilities.

- You've added your user to the group `j4a-admin` associated with the previously made subscription in the IAS tenant.
- Your system needs to be configured so that it trusts server certificates according to the "required" section of the BTP Trust Store .
- Your system needs to use a client certificate issued by a CA listed in the "required" section of the BTP Trust Store . One way to obtain a client certificate can be found on Manage Destination Certificates (under "Generate a Certificate").
- You've successfully applied SAP Note 3627794.
- Your ABAP system needs to support TLS 1.3. If TLS 1.3 is not enabled, refer to SAP Note 3318423 to activate it.

Procedure

Step 1 describes how you maintain the technical connection in your ABAP system, and steps 2 and 3 explain how you register your certificate in the SAP BTP Cockpit and in your ABAP System.

1. In your ABAP system, open the transaction `APPLDESTCC`.

    a. Select scenario `HTTP - Application Destination`, and click on **Execute**.

    b. Next, select `AIC_ADT_HTTP_PROXY` and open the context menu. Choose **Create New Application Destination**. Now, you're asked to enter the details of your application destination.

    c. Enter an **Application Destination Name** of your choice and select the check mark.

    d. Now, first enter a **Description** and select the checkboxes **Active** and **Default**.

    e. Fill in the information in the **Technical Settings** tab:

    - Target System Settings: In the **Host** field, enter `https://proxy.<your region>.hub.j4a.services.sap`. As **Protocol**, select **HTTPS**.

        > **Note**
        >
        > The region you enter in `https://proxy.<your region>.hub.j4a.services.sap` must be the same region (not technical key) that you chose for your subaccount, for example "eu10". See Regions and API Endpoints for the ABAP Environment.

    - (Optional) HTTP Proxy Options: This information is not mandatory.

    f. Now, choose the following in the **Logon** tab:

    - General Settings: Select **Connection to External Server**.
    - Logon Procedure: Select **Client Certificate** and choose the certificate you wish to use from the dropdown menu.

    g. Select **Save**.

    You've successfully maintained the connection.

2. Now, continue in your subaccount in the SAP BTP Cockpit.

    a. Select the SAP Joule for developers, ABAP AI capabilities subscription you have just created (see Subscribing to SAP Joule for Developers, ABAP AI Capabilities), or open its details and select **Go to application**. You are forwarded to the **Maintain Certificate Mappings** page.

    b. In the **Certificate Mappings** section, select **Create** to receive a one-time password (OTP). Copy the OTP.

    > **Note**

The OTP is valid for five minutes. If you need a new one, close the pop-up and click on **Create** again.

3. Next, go to your corresponding ABAP system and run report `AIC_ADT_PROXY_CERTIFICATE`. Select **Execute**.

    a. Paste the one-time password that you've copied in the previous step.

    b. (Optional) If you wish to use a new certificate that you haven't used in step 1, select **Use Custom SSL Client PSE ID**. Click in the **Client PSE ID** field and choose the new certificate from the value help. Now, you can go back to step 1 and switch to the new certificate.

    c. Select **Execute**.

You've successfully registered your certificate.

Results

You can now use SAP Joule for developers, ABAP AI capabilities in your ABAP system. For more information, see Capabilities.

Quick Start

If you're eager to get started, here's a short instruction that shows you how to access SAP Joule for developers, ABAP AI capabilities to try out its capabilities.

Context

You can access SAP Joule for developers, ABAP AI capabilities via the fully integrated Joule context menu or Joule view in ABAP development tools for Eclipse. Whether you have selected a specific development object or lines of source code, you can access the SAP Joule for developers, ABAP AI capabilities using the context menu or by searching for the Joule view. The predictive code completion is exclusively accessible via the toggle in the toolbar menu, and is automatically turned on by default.

Procedure

1. To open the Joule view, select **Window** > **Show View** > **Joule**.
2. Access the SAP Joule for developers, ABAP AI capabilities, such as the Explain capability, ABAP Unit Test Generation or CDS Test Generation, via the context menu from:
    - the Project Explorer
    - the source code editor (by selecting lines of code)
    - other views, such as the Outline or Search view
    - the Transport Organizer

Results

Once you have accessed the SAP Joule for developers, ABAP AI capabilities, you can interact with them in the Joule chat view using natural language to communicate your questions and requests. Whether you would like some explanations, want to generate tests, or need source code examples, SAP Joule for developers, ABAP AI capabilities supports you with any development-related topic.

Related Information

Capabilities

Learn

The content in **Learn** provides an overview and background information about SAP Joule for developers, ABAP AI capabilities, and helps you understand the concepts behind it. In addition, learn about recommendations and constraints when using SAP Joule for developers, ABAP AI capabilities.

Capabilities

Get a high-level overview of SAP Joule for developers, ABAP AI capabilities.

This section outlines the key features of SAP Joule for developers, ABAP AI capabilities, which are natively integrated in ABAP development tools for Eclipse. These capabilities include the following AI-based capabilities:

- **Joule Chat**: Interact with the Joule chat using natural language for asking questions or writing prompts. For details, see Chat.
- **Documentation Chat**: Use this capability to chat with public documentation on the SAP Help Portal. For details, see Documentation Chat.
- **Explain Capability**: Get detailed explanations of development objects or selected code by using the Explain feature. For details, see Explain.
- **Custom Code Migration AI Assistant**: The Custom Code Migration AI Assistant supports you with your custom code migration project. For details, see Using the Custom Code Migration AI Assistant.
- **Predictive Code Completion**: Leverage AI to predict code and to make suggestions for code completion. For details, see Predictive Code Completion.
- **ABAP Unit Test Generation**: The AI-based ABAP Unit features are specifically designed to assist you in creating and improving your unit tests more effectively. For details, see ABAP Unit Test Generation.
- **CDS Test Generation**: Use SAP Joule for developers, ABAP AI capabilities to boost your efficiency in generating unit test classes for ABAP CDS entities. For details, see CDS Unit Test Generation.
- **OData UI Service from Scratch**: Use the **OData UI Service from Scratch** in order to create all RAP-service related repository objects from scratch with the aid of the Joule chat. For details, see OData UI Service from Scratch.
- **RAP Business Logic Prediction**: Generate the implementation of a RAP determination or validation with the aid of the Joule. For details, see RAP Business Logic Prediction.
- **Consume Capability**: The Consume capability enables you to use the **OData Client Proxy** in order to consume a remote OData service. For details, see Consume.
- **GenAI-Driven Extensibility Assistant**: The extensibility assistant helps you accomplishing various tasks related to field extensibility like creating custom fields, finding business contexts, finding usages, finding value help views, and enabling business scenarios. For details, see Extensibility Assistant.

Explore each AI-based capability to optimize your use of SAP Joule for developers, ABAP AI capabilities.

Related Information

About the ABAP Development Tools: User Guide

Chat

The Joule chat capabilities allow you to converse in natural language to ask questions or obtain detailed explanations for code snippets or development objects.

About the Joule Chat

The Joule chat serves as your AI pair programmer, supporting you collaboratively in your everyday work as a developer. You can converse with the Joule chat in natural language, asking it any development-related questions and tasks. The Joule chat is fully integrated in the ABAP development tools for Eclipse and provides, for example, specific information about:

- General questions related to ABAP development or other programming languages.
- Specific questions about currently opened development objects or selected lines of code.
- General coding inquiries and the generation of code examples for specific development contexts.

For further specialization, the Joule chat features different topics with integrated skills to implement a broader range of tasks.

Overall, the chat capability leverages large language models to provide dynamic and contextually relevant conversations. It supports diverse development use cases by offering assistance in coding, debugging, explaining complex concepts, and providing recommendations based on best practices.

Get Started with the Joule Chat

To get started with the Joule chat, see Using the Joule Chat.

Documentation Chat

The documentation chat is a capability in SAP Joule for developers, ABAP AI capabilities. It allows you to chat with documentation on the SAP Help Portal.

About the Documentation Chat

The documentation chat accesses the documentation from the following guides:

- ABAP Keyword Documentation
- ABAP RESTful Application Programming Model
- ABAP Data Models
- ABAP Cloud

The chat considers your current project context and your documentation chat history. It provides answers based on the documentation listed before. The chat also provides you a link to the documentation to find the reference of the answer.

Learn about how to use the documentation chat in Using the Documentation Chat.

Predictive Code Completion

The predictive code completion enables you to use AI-based code completion in the ABAP source code editor. This capability allows developers to automatically generate ghost text to predict code, enhancing efficiency in writing code.

About the Predictive Code Completion

You can incorporate ghost text in your source code during your writing process, for example while writing code, comments, or ABAP Doc comments. Ghost texts are predicted based on your cursor position and the content in the ABAP source code editor. The predictive code completion currently supports the following object types:

- Interfaces
- Classes
- Programs

Get Started with the Predictive Code Completion

If you want to learn how to use the predictive code completion, see Using Predictive Code Completion.

Disclaimer

Note

When the predictive code completion feature is switched on, suggestions are shown as ghost texts. These suggestions are machine-generated without any human influence.

The following information describes how the underlying AI model has been trained.

Training data:

SAP has fine-tuned the StarCoder 2 model using internal SAP data. This data primarily consists of ABAP source code.

IP and personal data:

SAP retains intellectual property rights over the fine-tuning data.

Personal information has been filtered out to ensure compliance with privacy standards. No third-party data has been used during the fine-tuning process.

IP and personal data:

Robust security measures protect both the model and the data. Data is stored in a secure, access-controlled environment, with strong access controls only accessible by authorized individuals.

Explain

The Explain capability enables you to get detailed explanations of selected development objects or lines of source code.

About the Explain Capability

The Explain capability is fully integrated within ABAP development tools for Eclipse, supporting you in your development by providing detailed explanations of development objects and lines of source code. The Explain capability leverages large language models that are specifically trained to provide concise explanations of complex concepts, with special support for

- ABAP classes,

- ABAP programs,
- ATC findings (only simplification items),
- and CDS artifacts.

Currently, multiple object selection is not supported.

Starting the Explain Capability

The AI-based explanations of development objects are enabled for editors and views and can be accessed through various starting points in your IDE:

- Project Explorer
- Source code editor
- Joule chat view
- Other views such as the Outline, Relation Explorer or Search view

To get started with the Explain capability, see Using the Explain Capability.

Related Information

Using the Explain ATC Findings Capability

Custom Code Migration AI Assistant

The Custom Code Migration AI Assistant supports you with your custom code migration project.

About the Custom Code Migration AI Assistant

The Custom Code Migration AI Assistant in SAP Joule for developers, ABAP AI capabilities will be your starting point for understanding issues that arise along the way of converting your custom code from an ECC system to an SAP S/4HANA system. More specifically, it'll support you with your conversion by explaining SAP S/4HANA-related ABAP Test Cockpit (ATC) findings to you in detail, including the corresponding simplification notes, and can, on top of that, access a large pool of simplification items that can affect custom code and answer your questions about these items. Issues described can be:

- Data model and data type changes
- Functionality no longer available
- Incompatible changes made to SAP development objects

Starting the Custom Code Migration AI Assistant

Depending on your use case, there're different ways to open the assistant:

1. In the Joule chat window, select or type in /docs to start chatting about simplification items related to Custom Code Migration.
2. To chat with the assistant about SAP S/4HANA simplification items and Custom Code Migration, just start asking topic-related questions.

3. To get explanations of ABAP Test Cockpit (ATC) findings and to understand the issue described in the related SAP Simplification Item, you can open the SAP Joule chat directly via the finding.

4. To get explanations of ABAP programs, right-click your program and choose **Explain**.

To get started with the Custom Code Migration AI Assistant, see Using the Custom Code Migration AI Assistant.

Related Information

Using the Custom Code Migration AI Assistant
Using the Explain Capability

ABAP Unit Test Generation

SAP Joule for developers, ABAP AI capabilities provides ABAP developers with AI-based ABAP Unit features integrated into the ABAP development tools for Eclipse. These features are specifically designed to assist you, the ABAP Unit test author, in creating and improving your tests more effectively.

The following features can support you in the process of creating and improving ABAP Unit tests:

Feature	More Information
Test Generation	You can generate ABAP Unit tests from various locations within the ABAP development tools for Eclipse. For more information, see Test Generation.
Dependency Analysis	Identifying test-unfriendly dependencies helps you rethink and improve your unit tests. For more information, see Dependency Analysis.
Test Double Support	After identifying test-unfriendly dependencies, you can isolate them and use test doubles as replacements. For more information, see Test Double Support.
Test Code Explanations	Gain insights with test code explanations that help you identify test smells and understand test patterns better. For more information, see Test Code Explanations.
Generated Test Code Refactoring	As you enhance your test classes with new test methods, refactoring becomes essential. Explore various options to improve the quality of your test code and adjust it to meet your style preferences. For more information, see Generated Test Code Refactoring.
Splitting Test Classes and Test Methods	Improve your code structure by splitting test classes and test methods. For more information, see Splitting Test Classes and Test Methods.

Test Generation

With SAP Joule for developers, ABAP AI capabilities, you can easily access AI-based features designed to help you create ABAP Unit tests.

Scope

You can generate ABAP Unit tests for:

- Public, protected, and private methods of global ABAP classes
- Public methods of local classes within global class pools

Generating Unit Tests in ABAP Development Tools for Eclipse

In ABAP development tools for Eclipse, you have multiple starting points for generating unit tests. You can start the generation process from the following locations:

- Project Explorer
- Outline view
- ABAP Coverage view
- Using quick replies in the ABAP Unit chat in **Joule**. See Starting the ABAP Unit Chat.
- Using freestyle user prompts in the ABAP Unit chat in **Joule**. See Using Freestyle User Prompts.

To learn how to generate unit tests from the context menu for selected methods, see Generating ABAP Unit Tests.

General Principles

Before you start generating unit test methods, it's essential to understand some core principles. Understanding these will help you maximize the effectiveness of your testing efforts. Here are a few key points to keep in mind:

- **Understand the problem domain:** Since Joule doesn't know what the tested code should do, it's crucial for you to have a clear understanding of the problem domain.
- **Start small:** Focus on one to three productive methods to test. This approach limits the amount of generated code, likely leading to better code quality overall. Starting small allows you to refine and improve your tests more effectively.
- **Review and decide carefully:** Thoroughly review the generated test code to ensure that it aligns with the specification of the tested code. Keep the tests if they match; otherwise, make necessary adjustments
- **Customize with user prompts:** You can manually adjust the code or ask Joule for assistance. For example, you might say, “Use inline declarations” or “Avoid using Hungarian notation for variable names”. This customization is important because it helps tailor the code to your specific needs and preferences.
- **Improve future code suggestions:** Making manual adjustments and corrections to the code that Joule has previously generated enhances the quality of future proposals. When you request Joule to create additional tests, it will review existing tests and offer new test methods in a similar style.
- **Run your unit tests:** Each time you modify the code—whether it's the productive code or the test code—remember to run the entire test suite.
- **Resolve failing unit tests:** If any unit tests fail, prioritize resolving them before proceeding to other tasks. To do this, either modify the production code or adjust the test code to ensure all tests pass successfully.

Related Information

ABAP Unit Test Generation

Dependency Analysis

Use the AI-based dependency analysis feature in ABAP development tools for Eclipse to identify test-unfriendly dependencies.

General Idea

Understanding class dependencies is essential for identifying unstable dependencies, such as those involving databases or other APIs. These test-unfriendly dependencies may return unexpected test results depending on systems, users, or other aspects in your code. Therefore, it's useful to replace test-unfriendly dependencies with test doubles to enhance the stability of your tests.

How to Identify Test-Unfriendly Dependencies

You can efficiently identify test-unfriendly dependencies using an AI-based feature, which supports you throughout the process. For more detailed guidance on how to use this feature, see Analyzing Dependencies.

Related Information

ABAP Unit Test Generation

Test Double Support

After identifying any test-unfriendly dependencies in your test classes, isolate these dependencies by replacing them with test doubles to improve the stability of unit test results.

General Idea

By substituting calls to database tables or APIs with test doubles, you ensure that your unit tests remain stable and predictable, regardless of the system you're working on.

Adding Test Doubles in ABAP Development Tools for Eclipse

To learn how to resolve test-unfriendly dependencies by adding test doubles, see Adding Test Doubles.

Related Information

Using Freestyle User Prompts
Dependency Analysis
ABAP Unit Test Generation

Test Code Explanations

Use the AI-based explain feature for ABAP Unit to specifically understand your unit tests in more details.

Use Cases

The explain feature provides detailed answers to these key questions:

- **Test scope**: What is the tested scope? What is the general purpose of your test class?
- **Test smells**: Are there any test smells in your test class? A common example is test code duplication.

- **Test patterns**: Which test patterns have you implemented in your code? Using creation methods or custom assertions are typical examples.

How to Use

If you want to learn how to use this feature in ABAP development tools for Eclipse, see Getting Test Code Explanations.

Related Information

ABAP Unit Test Generation

Generated Test Code Refactoring

When you use AI to generate unit tests, it's important to refactor the code as part of your process. By using AI-based refactoring instructions, you ensure that your test code meets the quality standards you require and also fits your preferred coding style.

Refactor Your Generated Code with These Instructions

You can use predefined instructions, or type your own in the chat, to refactor your generated code effectively. Consider the following suggestions:

- Avoid hungarian notation (no prefixes)
- Create CUT in setup method
- Extract and use parameterized custom assertion
- Inline and remove custom assertions
- Program against interfaces instead of classes
- Use inline declarations
- Use constructor expressions to fill structures and tables
- Write assert statements in one line

How to Refactor Generated Test Code with ABAP Development Tools for Eclipse

If you want to learn how to refactor your generated test code with ABAP development tools for Eclipse, see Refactoring Generated Test Code.

Related Information

ABAP Unit Test Generation

Splitting Test Classes and Test Methods

Improve your code structure by splitting test classes and test methods.

Splitting Test Classes

When you split a test class into two separate test classes and one helper class, you can organize your tests thematically and reduce code duplication.

To learn how to split test classes with ABAP development tools for Eclipse, see Splitting Test Classes.

Splitting Test Methods

Consider splitting your test method in the following situations:

- When it tests different aspects instead of focusing on a single function.
- It gets too large.
- It includes too many assertions.

To learn how to split test methods with ABAP development tools for Eclipse, see Splitting Test Methods.

Related Information

ABAP Unit Test Generation

CDS Unit Test Generation

The AI assisted test class generates automated tests for ABAP CDS entity. This helps in generating more context-specific content in the ABAP unit tests for a CDS entity. Hence, providing basic test coverage and improving developer efficiency.

Testing CDS Entity

CDS Test Double Framework enables you to test the logic expressed in CDS entities in an automated way. To test the logic in CDS entity under test (CUT), test data must be inserted in the double. This data is returned by the test doubles when the CUT is executed. CUT must not be modified by the test framework to enable unit or hierarchical testing.

CDS Test Double Framework enables unit or hierarchical testing of CDS views by automatically creating updatable test doubles for each dependent component in the same database schema. For more information, see ABAP CDS Test Double Framework.

Get Started with CDS Unit Test Class Generation

If you want to learn how to use the CDS Unit Test Class Generation, see Using CDS Unit Test Class Generator.

OData UI Service from Scratch

You can create all RAP service-related repository objects from scratch by using the **OData UI Service from Scratch** wizard.

About the OData UI Service from Scratch

The **OData UI Service from Scratch** wizard allows you to create a RAP BO without the requirement of an already existing artifact and without having to manually create all the necessary repository objects yourself. The only prerequisite for using this wizard is an already existing package. You can then use the Joule chat to define the requirements for your BO.

Get Started with the OData UI Service from Scratch

To get started with the OData UI Service from Scratch wizard, see Using the OData UI Service from Scratch Wizard with the Joule Chat.

RAP Business Logic Prediction

The RAP business logic prediction supports you in implementing determinations or validations in the implementation class of your behavior definition.

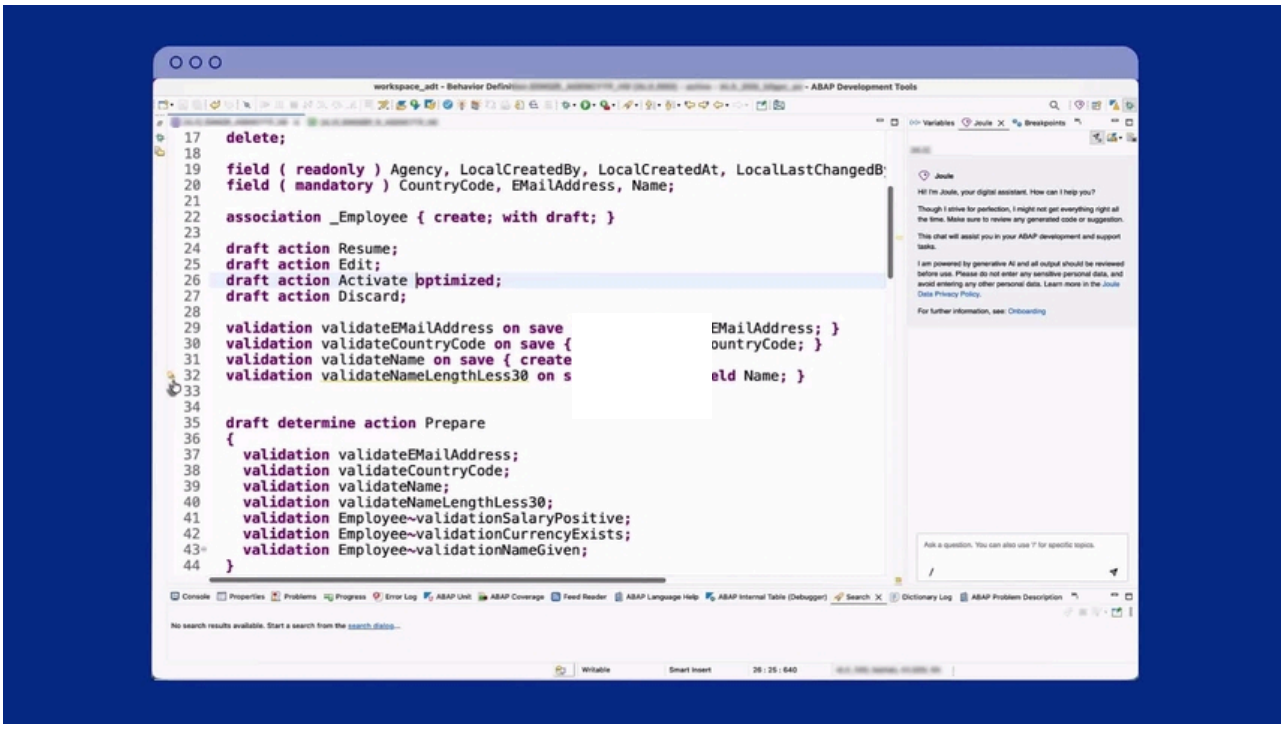
About the RAP Business Logic Prediction

This AI-based capability supports you with the implementation logic for your behavior definition that contains a method with either a determination or validation logic. This capability enables AI-based generation of the implementation code for the specified determination or validation.

As the foundation for Joule's code generation, you can define a method description in natural language detailing the purpose and checks performed by the determination or validation. The RAP business logic prediction uses this description as a guiding principle to generate a more complex implementation of a RAP determination or validation.

Video Tutorial

RAP Business Logic Prediction



Open this video in a new window

Get Started with the RAP Business Logic Prediction

To get started with the RAP business logic prediction, see Using RAP Business Logic Prediction.

Consume

The **Consume** capability enables you to use the **OData Client Proxy** in order to consume a remote OData service.

About the Consume Capability

Joule offers you the option to generate both the OData requests and the ABAP coding. Both can be used to consume a corresponding OData call via the OData Client Proxy. You simply provide a prompt explaining what you want to consume. For example, you can write "I want to read the field ID from all campaigns" in the Joule chat using the **/consume**. Joule then responds with the corresponding request and ABAP code.

Get Started with the Consume Capability

To get started with the Consume capability, see Using the Consume Capability.

Embedded Analytics Star Schema Generator Powered by AI

The CDS Analytical Model Generation powered by AI is a tool used to help create an analytical star schema using AI.

The creation of an analytical star schema is the basis for realizing analytical scenarios. Using the CDS Analytical Model Generation wizard powered by AI, you are able to derive an analytical model based on CDS view entities.

Based on the root CDS view entity, the following elements are generated by default:

- **Cube definition:** the center of a the star schema, the cube view is a CDS view entity annotated with the annotation `@Analytics.dataCategory: #CUBE`. A cube consists of dimension fields with associations to dimension views and measures.
- **Dimension fields** (like Product, Region, Time, Customer) that categorize and provide context.
- **Measure fields:** transactional data which is measurable.

Related Information

Embedded Analytics Star Schema Generator Powered by AI
Star Schema (ABAP Analytics Guide)

Recommendations and Constraints

This chapter gives a general overview of boundary conditions for SAP Joule for developers, ABAP AI capabilities.

General Constraints and Recommendations for Generative AI

Processes and features based on generative AI technologies are non-deterministic by definition. Even when all measures are taken to ensure only expected and valid results, the final result can't be guaranteed. For example, the same prompt can generate different responses from a large language model at different points in time, and hallucinations are possible with large language models.

→ Recommendation

We recommend that all AI-generated development objects and code snippets must always be reviewed for functional and semantic correctness by human developers.

SAP Joule for Developers, ABAP AI Capabilities

Capabilities and skills in SAP Joule for developers, ABAP AI capabilities rely on different large language models as part of their implementation. The used large language models can change at any point in time, which can lead to varying results using the same input parameters or prompts.

→ Recommendation

We recommend that you always verify that AI-generated development objects and code snippets fulfill your functional and non-functional requirements before using them as part of your productive code base.

Develop

The content in **Develop** is designed to guide you through the practical application of the AI features for SAP Joule for developers, ABAP AI capabilities. It offers hands-on tutorials, step-by-step instructions, and useful tips to help you effectively implement and utilize these AI capabilities in your projects.

Using the Joule Chat

Get support while developing in ABAP development tools for Eclipse by using the Joule chat for questions and tasks.

About the Joule Chat

The Joule chat acts as your AI pair programmer in ABAP development tools for Eclipse, assisting you in your daily work as a developer, particularly with ABAP-related questions and tasks. These inquiries should relate to programming in general. In addition to supporting your development work by answering general questions about your code or ABAP programs, the Joule chat can also assist you with specialized topics such as the **Explain** capability and **ABAP Unit Test Generation**.

IDE Context Awareness

The IDE Context represents the current state of your IDE. When the Joule chat is used, the IDE Context contains information from all the resources of an ABAP project that are currently open in the IDE. Each editor and view can contribute one or more resources they represent. This information is considered by the Joule chat when generating answers for your inquiry. If you have selected a specific development object, the Joule chat will list this object under **Used objects** as a link before providing its answer. This IDE context awareness ensures that you are always informed about which development objects have influenced the Joule chat's responses.

Topics Covered in the Joule Chat

Topics are specializations of the different capabilities, which implement tasks that are tailored for different development objects or use cases, such as explaining development objects or creating CDS and ABAP unit tests. The Joule chat covers the following topics, which can be accessed through slash commands:

Topics Covered in the Joule Chat

Topic	Slash Command	Description	More Information
Joule chat default		The Joule chat is the default option and responds when no other topic is selected. It responds to any development-related questions or requests.	For more information, see Chat.

Topic	Slash Command	Description	More Information
Explain	/explain	The **Explain** provides detailed, semantic explanations about the technical contents of development objects and selected source code.	For more information, see Explain.
ABAP Unit Test Generation	/aunit	The **ABAP Unit Test Generation** supports you with different ABAP Unit capabilities, for example, creating unit tests for your ABAP classes to check their functional correctness. Other use cases include: • Analyzing dependencies • Adding test double support • Getting test code explanations • Refactoring generated test code	For more information, see ABAP Unit Test Generation.
Consume	/consume	The **Consume** enables you to use the OData Client Proxy in order to generate OData requests and/or ABAP code for consuming a remote OData service.	For more information, see Consume.
Documentation Chat	/docs	The documentation chat allows you to chat with public documentation on the SAP Help Portal.	For more information, see Documentation Chat.

Related Information

Opening the Joule Chat View
Working with the Joule Chat
Using the Explain Capability
Using the ABAP Unit Test Generation Capability

Opening the Joule Chat View

Learn how to open and navigate the Joule chat view in ABAP development tools for Eclipse.

Context

The Joule chat is an integrated view in ABAP development tools for Eclipse, offering multiple support options for your development work through the interaction with an AI in natural language. Use this step-by-step procedure to open the Joule chat view and familiarize yourself with its topics.

Procedure

1. To open the Joule chat view, choose Window > Show View > Other > Joule from the main menu.

   → Tip

   You can also open the Joule chat view by choosing Open Joule View ( ) from the main toolbar.

2. The Joule chat view opens with Joule introducing itself as your digital assistant. Joule also states the data usage guidelines and reminds you to always review any generated code or suggestions.
3. Project Selection: The current project for the chat conversation is displayed in the top left corner. To change the project, select Change project for chat in the view toolbar of the Joule chat view.
4. Type your message in the input field at the bottom of the chat view. To begin a new line, simply use Shift + Enter .
5. Select Send message or press Enter to send the message.
6. Your message and Joule's answer will appear in the chat history.

   ⓘ Note

   If Joule's answer takes too long, you can stop the process by selecting the Stop current request button that appears in the view during the processing of your question.

7. Specific Topic Selection: The / button enables you to select a specific topic, such as /explain for the Explain capability or /aunit for generating ABAP unit tests.
8. To clear the chat history, choose Clear Chat from the view toolbar. This resets the entire conversation.

   ⓘ Note

   The Joule chat history is not saved and will be lost once you close and reopen ABAP development tools for Eclipse.

Related Information

Working with the Joule Chat
Using the Explain Capability
Using the ABAP Unit Test Generation Capability

Working with the Joule Chat

Get additional information on how to chat with Joule in ABAP development tools for Eclipse.

Using the Joule chat is a collaborative process. As the developer, you can, for example, ask Joule to be more precise, less detailed, or to improve or change your source code.

Example Questions and Prompts

The way you phrase your request will influence Joule's answer. You can, for example, ask the Joule chat to:

- Be brief.
- Summarize in 3 sentences.
- Provide a detailed answer with source code examples.

You can ask general or specific questions. Here are some examples:

- What is RAP?
- How can I create a new ABAP program?
- How can I generate ABAP Unit Tests?

Additionally, you can select lines of source code, for example a method, in the source code editor and then give your prompt:

- Propose a better name for this method.
- Generate annotations for the selected source code.

If you have any follow-up questions or tasks, you can give them as input after Joule's initial response, and it will consider your previous conversation to process your question or prompt. This makes it possible to refine and guide your conversation towards your goals.

ℹ Note

Even if you ask the Joule chat the same questions consecutively, the generated answers may differ. This is because the large language models used can change at any time, leading to varying results even with the same input parameters or prompts.

Joule's Answer and Chat History

The Joule chat can distinguish between different topics in the chat history. It considers both the chat history and IDE context when generating a new answer for a request.

- If Joule's answer takes too long, you can stop the process by selecting the **Stop current request** button that appears in the view during the processing of your question. Joule answers your questions or prompts step-by-step through its streaming mode and it can deliver regular status updates on its progress.
- If you are not satisfied with the ongoing conversation and would like to reset the chat to an earlier point in the history, you can use the **x** button located at the top-right corner of one of your written messages. This will delete all chat history below, including the message you've chosen.
- The chat can also suggest code examples and generates formatted code snippets that you can copy to the clipboard and use in your current source code editor using the **Copy** button on the top-right corner of the code snippet.
- Choosing **Insert** inserts the code snippet at the current cursor position or replaces the selected code in the active editor. This feature is only supported in source-based editors.
- If you have selected a development object or source code from the editor for one of your inquiries, the Joule chat view displays the development object before its answer in the **Used object** field.

Automatic Topic Dispatching

Joule chat lets you enable automatic dispatching of your message to the most suitable topic for handling its content. This feature is on by default. You can turn it off using the **Dispatch to Topic** button () in the view menu.

Here's how automatic topic dispatching works:

- Your typed message is directed to the topic that best matches its content. For example, if you want to create a unit test, your message is dispatched to the **/aunit** slash command.
- Your message can be dispatched to the following topics: **Explain** and **ABAP Unit Test Generation**. If neither topic applies to your message, the Joule chat default generates a response.

- Automatic topic dispatching is enabled only when no slash command (**/explain** or **/aunit**) is specified in your message. If you've already used a slash command, the corresponding topic generates the response.

Related Information

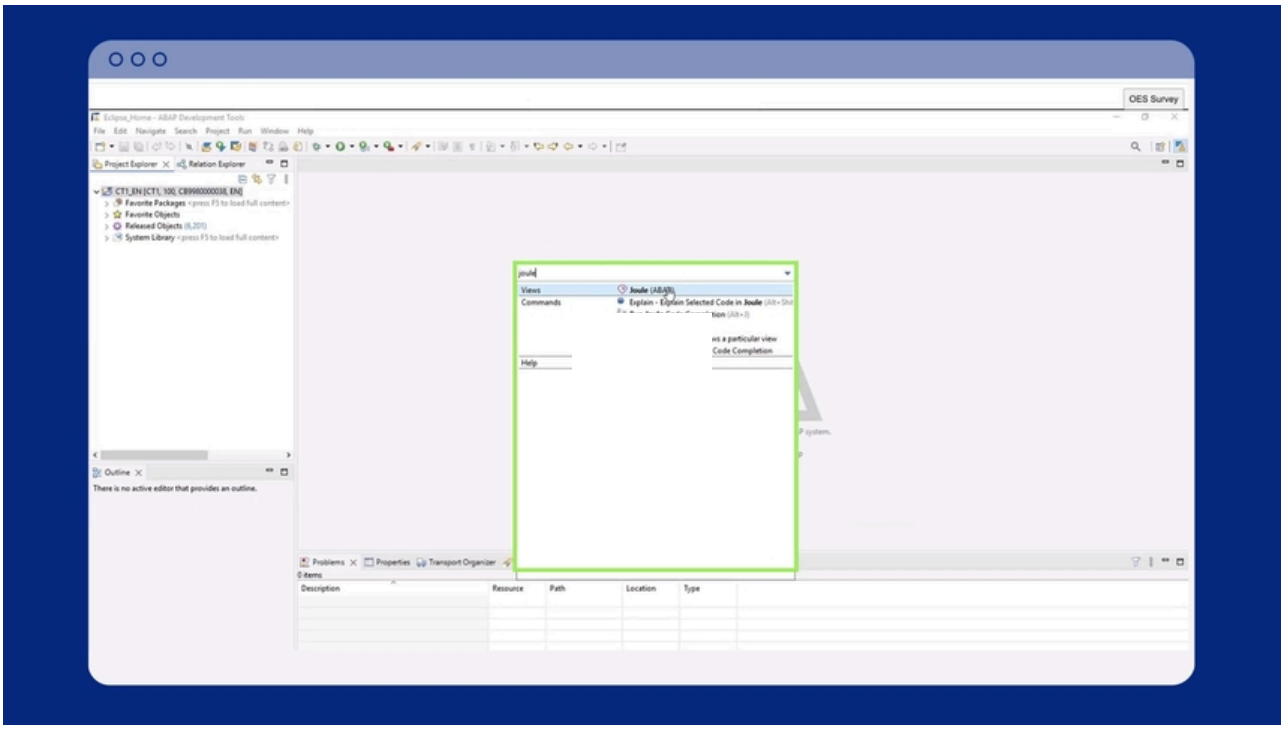
Video Tutorials for the Joule Chat

Video Tutorials for the Joule Chat

Learn more about how to use the Joule chat effectively and get an overview of its features with the following video tutorials.

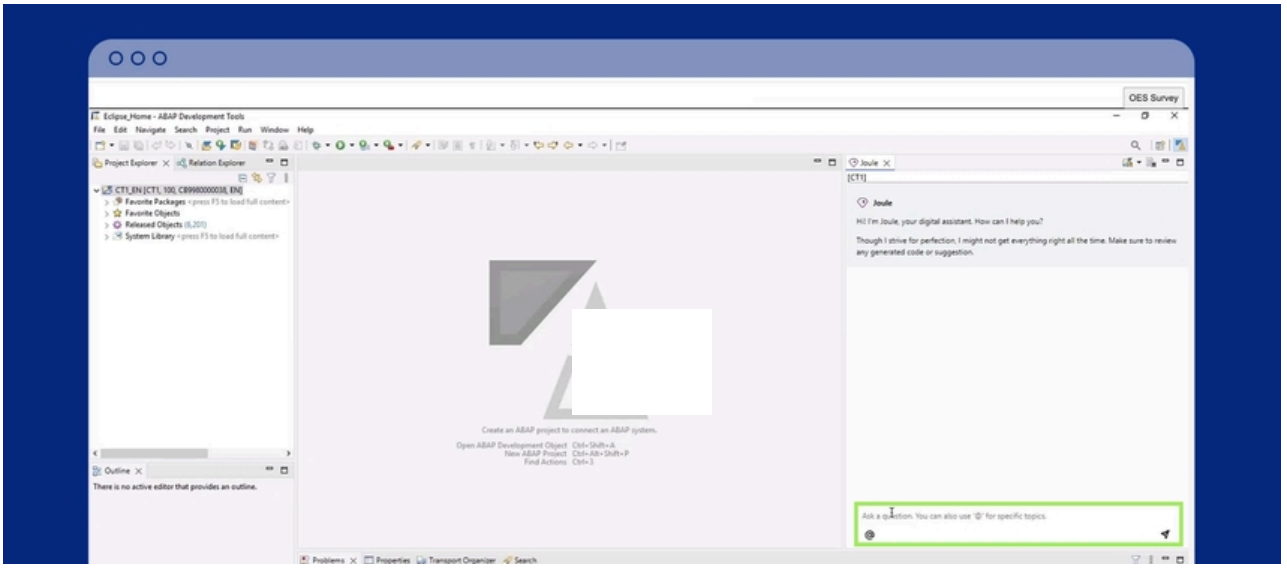
Video Tutorials

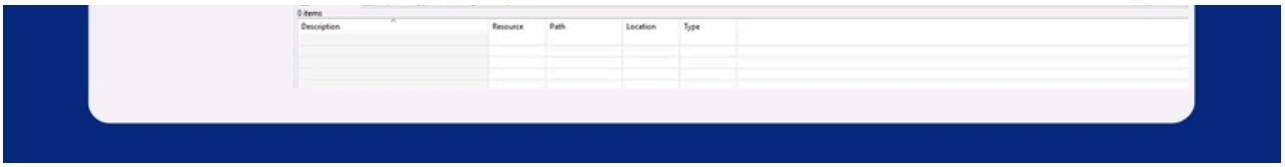
Opening Joule Chat



Open this video in a new window

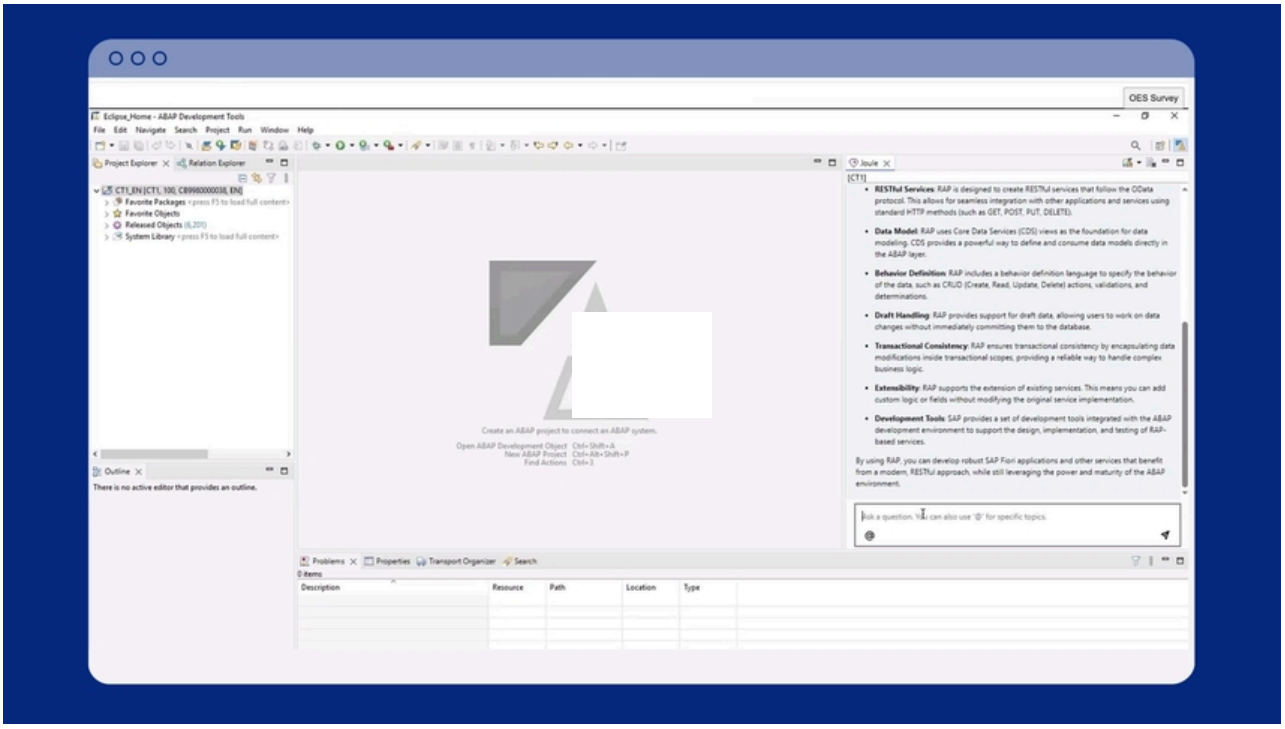
Details about the Joule Chat View





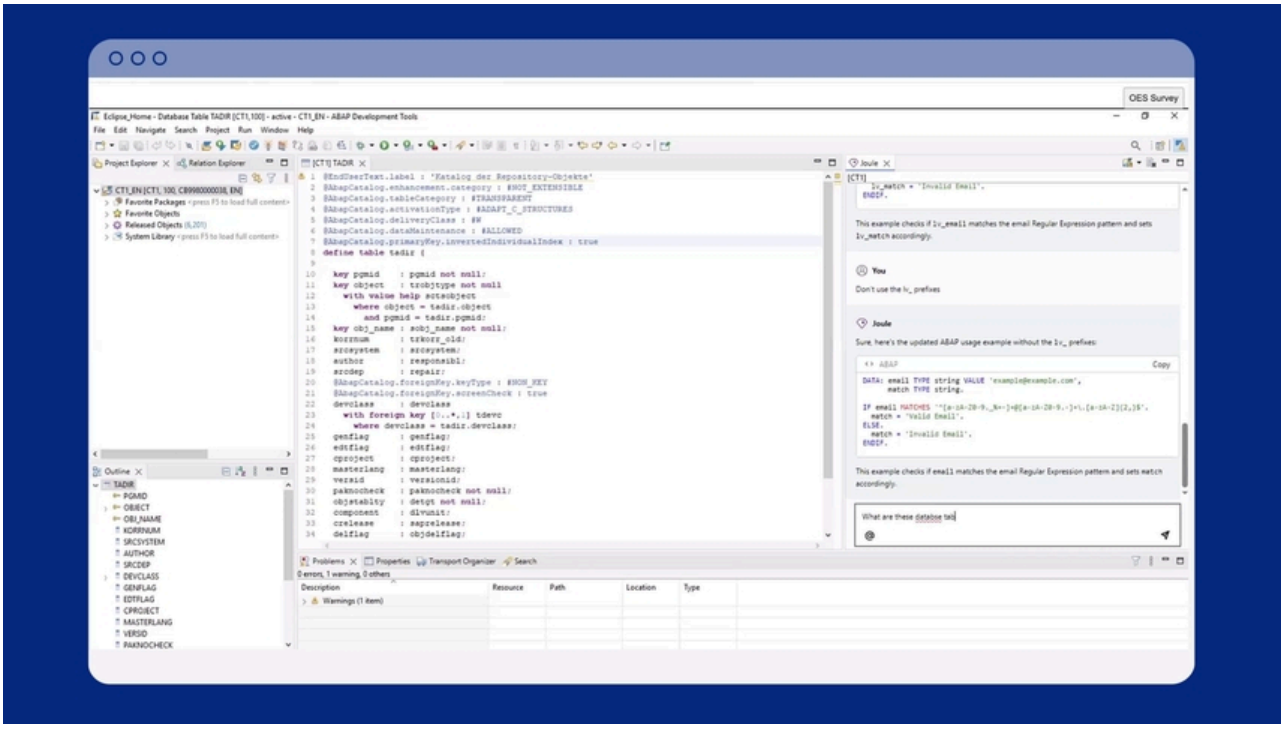
Open this video in a new window

Having a Conversation with Joule



Open this video in a new window

Paste Code for Explanation



Open this video in a new window

Using the Documentation Chat

Learn how to use the documentation chat.

For general information about the documentation chat, see Documentation Chat.

Prerequisites

You must have the Joule view installed in ABAP Development Tools for Eclipse.

For more information about the Joule view, see SAP Joule for Developers, ABAP AI Capabilities.

Procedure

In the **Joule** view, click the bottom-left slash (/) and select **docs**. You can also enter **/docs** manually. Always use **/docs** in front of your prompt. Enter your prompt and choose Enter. Examples of prompts include:

- What are determinations?
- List ABAP functions.
- How to dynamically create reports.
- Give me an example of an action.

If you choose **/docs**, don't enter a prompt and only click **Enter**, you receive an onboarding message with a brief description and two sample questions regarding the documentation chat.

Using the Explain Capability

Get detailed explanations while developing in ABAP development tools for Eclipse by using the Explain capability.

About the Explain Capability

Joule, as your AI pair programmer in ABAP development tools for Eclipse, offers you the option to get detailed explanations of a development object or lines of code. For example, it provides special support for

- ABAP classes,
- ABAP programs,
- ATC findings (only simplification items),
- and CDS artifacts.

When you use the Explain capability in ABAP development tools for Eclipse, the explanations automatically appear in the Joule chat view with the option of follow-up questions or, in some cases, quick replies that generate an even more detailed and more understandable explanation if needed.

IDE Context Awareness

The IDE Context represents the current state of your IDE. When the Explain capability is used, the IDE Context contains information from all the resources of an ABAP project that are currently open in the IDE. Each editor and view can contribute one

or more resources they represent. This information is considered by the Explain capability when generating an explanation for your inquiry. However, your selected object will primarily influence the Explain's answer.

Get Started

The Explain capability is natively integrated into ABAP development tools for Eclipse and can be accessed through various starting points. Learn how to open the Explain capability and discover best practices.

Related Information

Opening the Explain Capability
Working with the Explain Capability
Video Tutorials for the Explain Capability
Using the Explain ATC Findings Capability

Opening the Explain Capability

Learn how to open the Explain capability in ABAP development tools for Eclipse.

Context

When you use the Explain capability in ABAP development tools for Eclipse, its explanations will appear in the Joule chat view and are marked with **/explain** to distinguish them from the default Joule chat answers. Use this step-by-step procedure to access the Explain capability in ABAP development tools for Eclipse and familiarize yourself with its additional features.

Procedure

1. To open the Explain capability, select a development object from either:
    - the Project Explorer,
    - the source code editor (by selecting lines of code),
    - or other views such as the Outline, Relation Explorer or Search view.
2. Then proceed with either of the two following options:
    a. Choose **Joule** > **Explain** from the context menu.
    b. Open the Joule chat view by selecting **Window** > **Show View** > **Other** > **Joule** from the main menu and selecting **/explain** located in the input field at the bottom of the view.

Results

The Explain's answer will appear in the Joule chat view with the selected development object listed under **Used objects**. The Explain capabiltiy will also show any resources that were used to answer your questions under **Used objects**.

Example

When you run the Explain capability on a BDEF which offers a KTD documentation, both the BDEF and the underlying KTD object will be listed under **Used objects**.

Related Information

Working with the Explain Capability

Get additional information on how to use the Explain capability in ABAP development tools for Eclipse.

You can use the Explain capability anytime you need a more detailed explanation of a development object or source code.

Example Questions and Prompts

Using the Explain capability is a collaborative process. You can specify to the Explain capability in natural language how the explanation should be generated, for instance, whether it should be detailed or concise, with or without examples.

You can ask the Explain capability any development-related questions, or give prompts such as:

- How can the selected source code be enhanced? Give examples.
- Are there nested loops in the selected function module?
- Explain the use of this development object in a few sentences.
- Explain the annotations of this ABAP class.

IDE Context Awareness

The Explain capability is IDE context aware and considers this when generating an answer. This is illustrated in the following use case: If the Explain capability does not know for which selected object you would like an explanation, it will ask you which object you are referring to and allow you to select from a list of development objects through quick replies.

Used and Attached Objects

When you choose an object for the Explain capability, this object is automatically attached to both your message and the Explain's response. You can see the attached object link at the top of your own message or as the used object link in the Explain's response before the explanation. The same applies when you select lines of code in the source code editor.

ℹ Note

The Explain capability will also show any other resources that were used to answer your question under **Used objects**.

Quick Replies

For some development object types, you will get the option to guide the conversation to a specific outcome with quick replies. These quick replies will appear at the end of the Explain's response in the Joule chat view. For example, you may have the option to generate a more detailed or more understandable explanation through the use of such quick replies.

ℹ Note

Quick replies are not enabled for all development object types.

Working with the Explain Capability In-App

Within the Custom Code Migration use case, you can use the Analyze Custom Code/Custom Code Migration app to get explanations of your ATC findings, and your code, in-app. For more information on the app, see Custom Code Analysis.

This is how you use the in-app functionalities:

Code Explain

1. First, you'll need to define a communication arrangement for the communication scenario `SAP_COM_0464` to establish a connection between the Analyze Custom Code/Custom Code Migration app running in an SAP BTP ABAP environment, with your SAP S/4HANA or SAP S/4HANA Cloud Private Edition ABAP system using remote function calls (RFC). See SAP_COM_0464.
2. Define a communication arrangement for communication scenario `SAP_COM_0B22` for your cloud system to enable the Code Explain feature in the Anaylze Custom Code/Custom Code Migration app. See SAP_COM_0B22.
3. On the **SAP Fiori launchpad**, in the **IT Project Management** section, select the **Custom Code Migration** app.

   > **Note**
   >
   > If you have the `Quality Manager – Software Development` role instead of the `Project Manager – IT` role, you'll find the Analyze Custom Code app in the **Custom Code Analysis** tab.

4. Choose **Create**, then **SAP S/4HANA Migration Project**.
5. In the Project Data dialog, enter **Project Description**, **Transition Scenario**, **Material Number Length**, **Location of Custom Code**, and **Target Release**.
6. The value for the **Object Provider**, if **Location of Custom Code** is chosen, is the value of the object provider defined in `SAP_COM_0464`.
7. Choose **Create**.
8. Once the project is created, you can choose **Start Analysis Run** to initiate the analysis.
9. After the finished run, choose **Number of Findings**.
10. Select any of the findings at the bottom, then choose the **Code Explain** button to get an explanation of your code or the **ATC Explain** button to get an explanation of the ATC finding's simplification items.

    For the Code Explain, you have the option to scroll down in the pop-up window, where, at the very bottom, you'll find two links providing you with more detailed explanations on business purpose and processing logic.

    For the ATC Explain, in the pop-up window, you'll find two tabs at the top: **Explanation** and **References**. You'll find your finding explanation in the former and the source for the explanation in the latter tab.

ATC Explain

For this functionality, you won't have to define a communication arrangement for communication scenario `SAP_COM_0B22`. Just enable the usage of your Analyze Custom Code/Custom Code Migration app as described above.

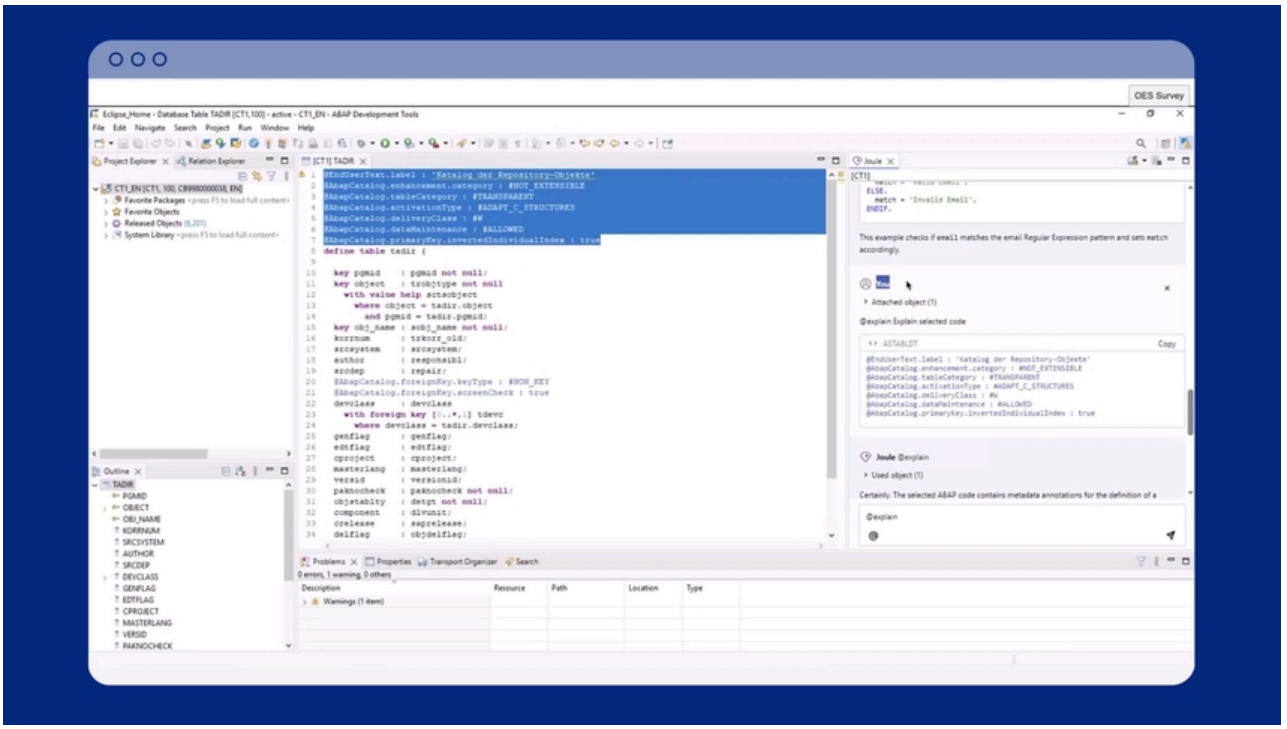
To learn more about AI capabilities for Custom Code Migration, see Using the Custom Code Migration AI Assistant.

Video Tutorials for the Explain Capability

Learn more about how to use the Explain capability effectively with the following video tutorials.

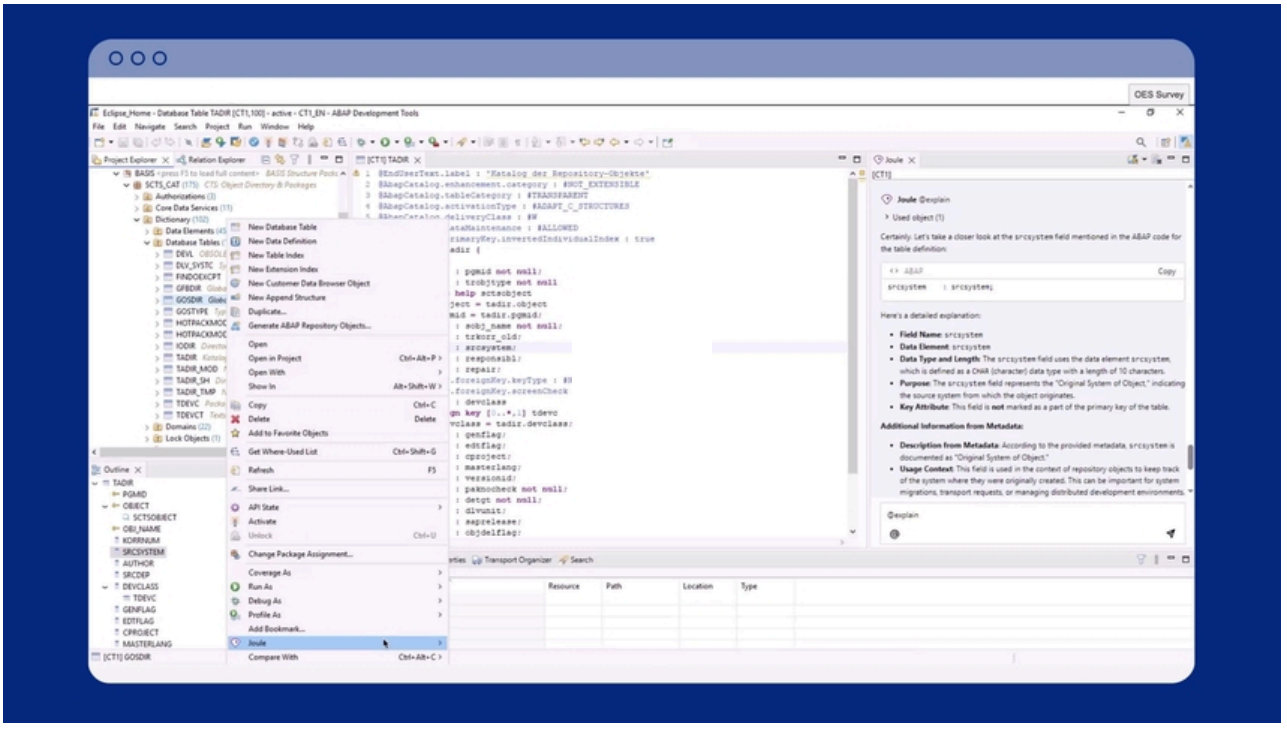
Video Tutorials

Running the Explain Capability for Lines of Code



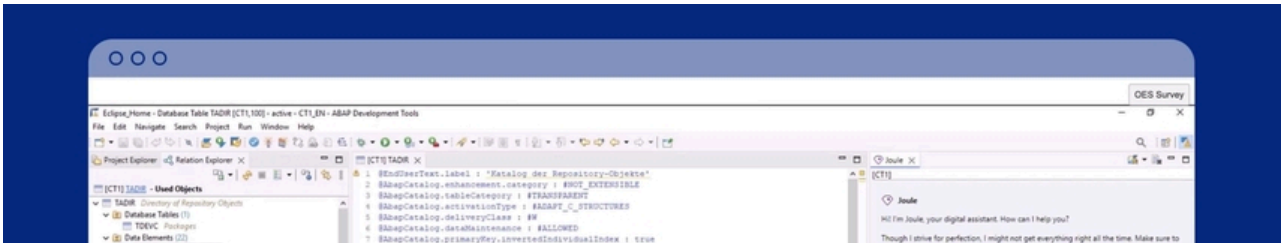
Open this video in a new window

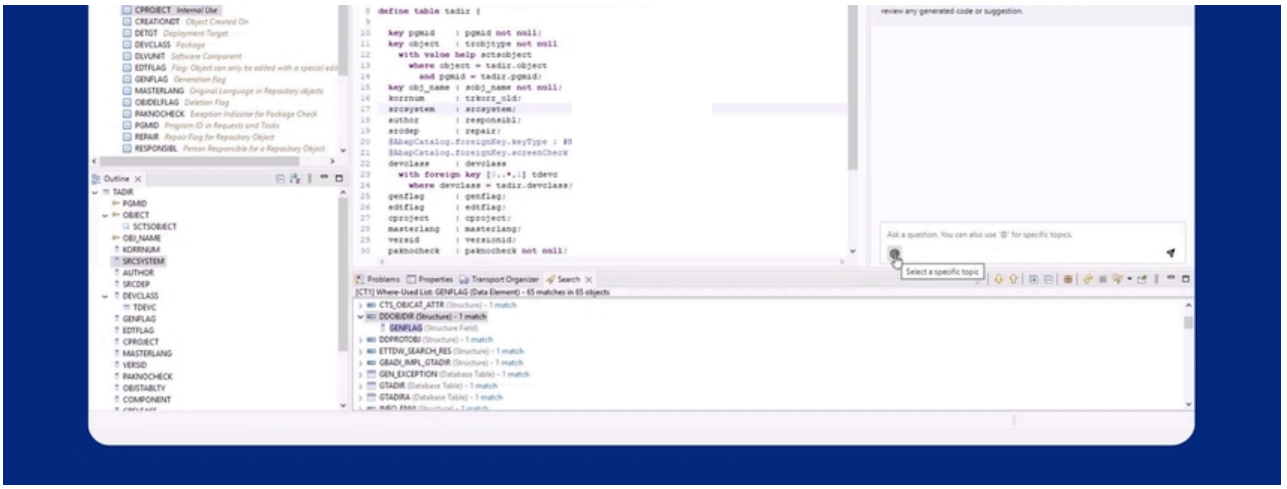
Running the Explain Capability for a Development Object in a View



Open this video in a new window

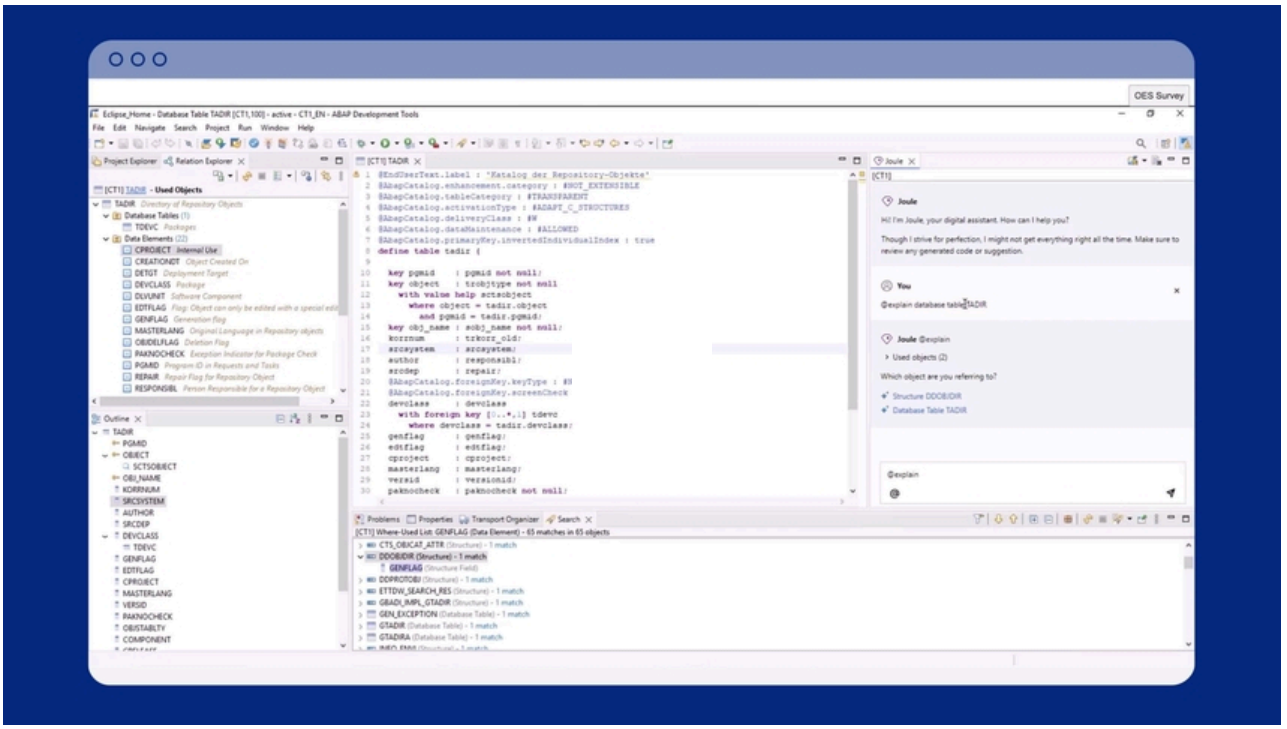
Selecting the Explain Capability in the Joule Chat





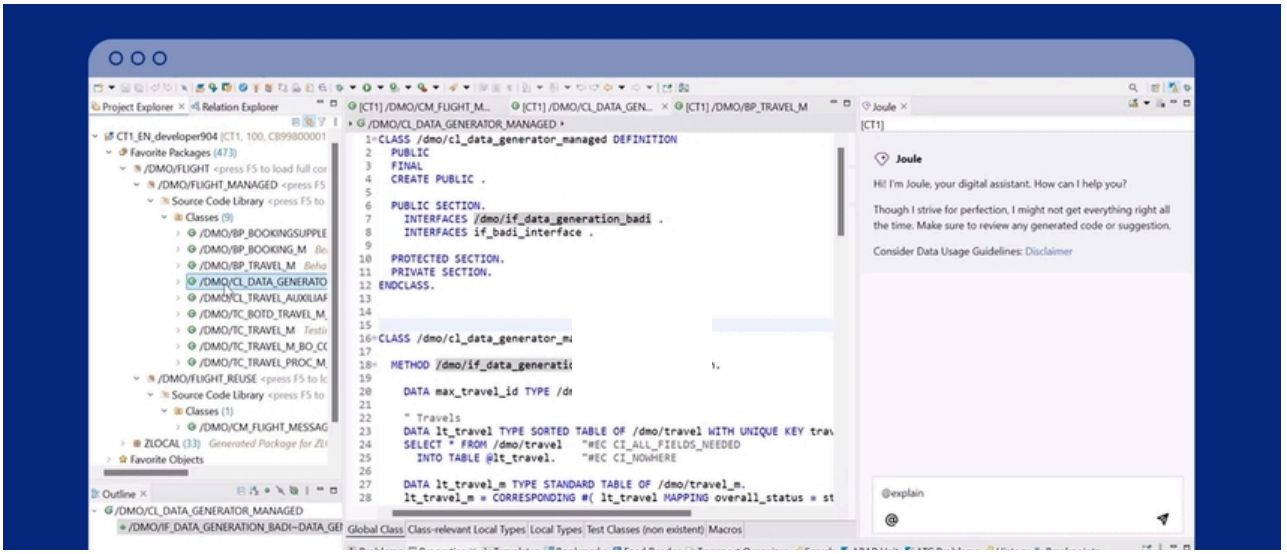
Open this video in a new window

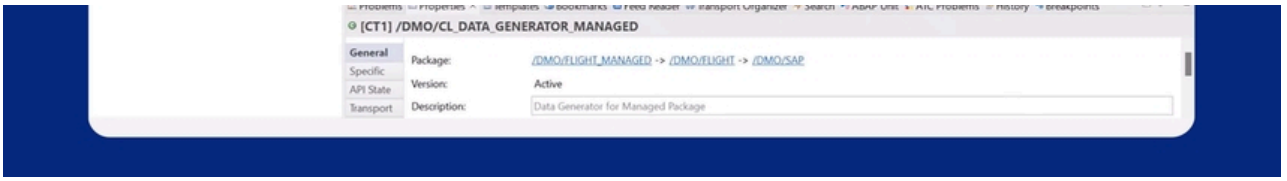
Running the Explain Capability from the Joule Chat



Open this video in a new window

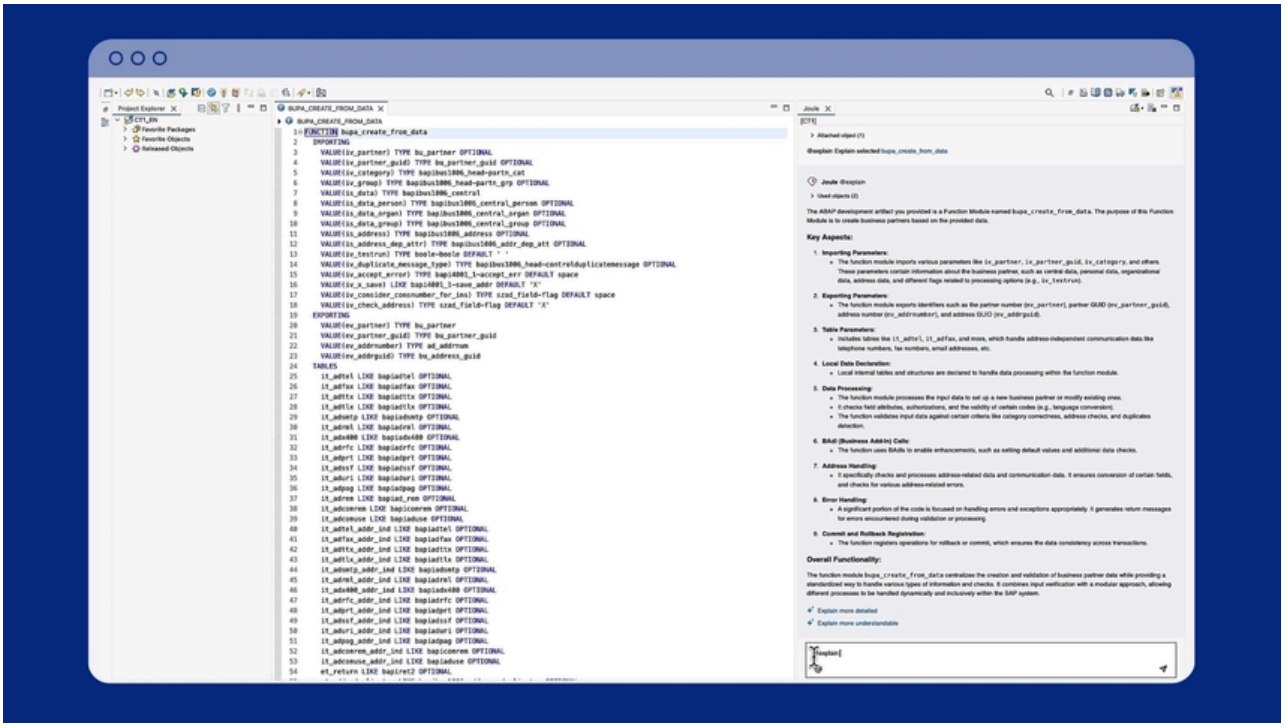
Running the Explain Capability for ABAP Code in Classes from the Project Explorer





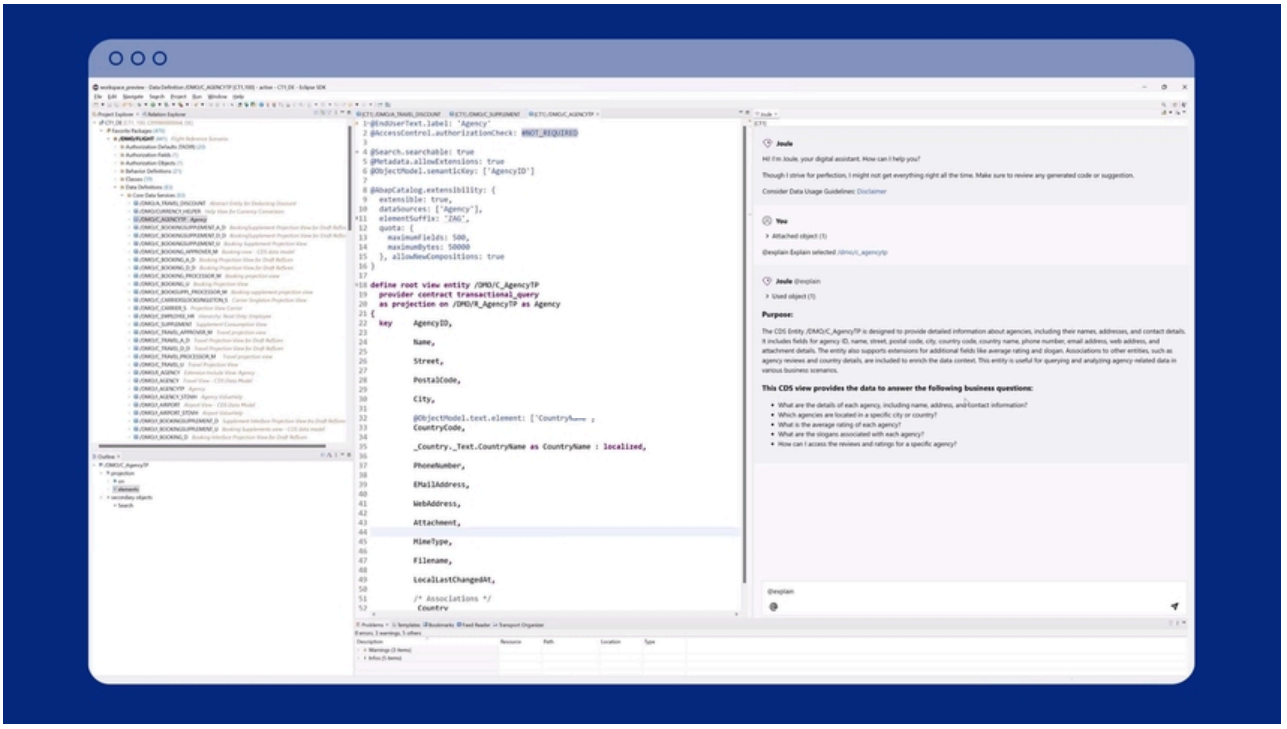
Open this video in a new window

Running the Explain Capability for ABAP Code in Classes from the Context Menu and Joule Questions



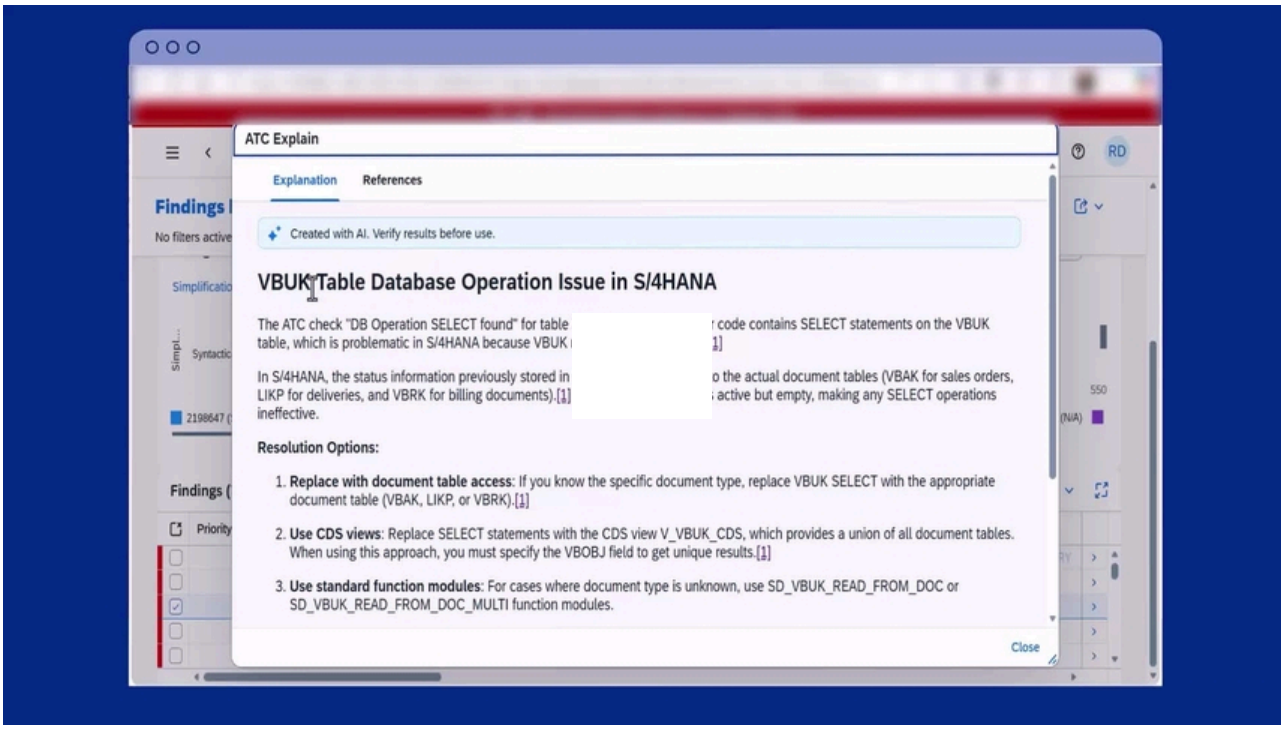
Open this video in a new window

Running the Explain Capability for CDS Views



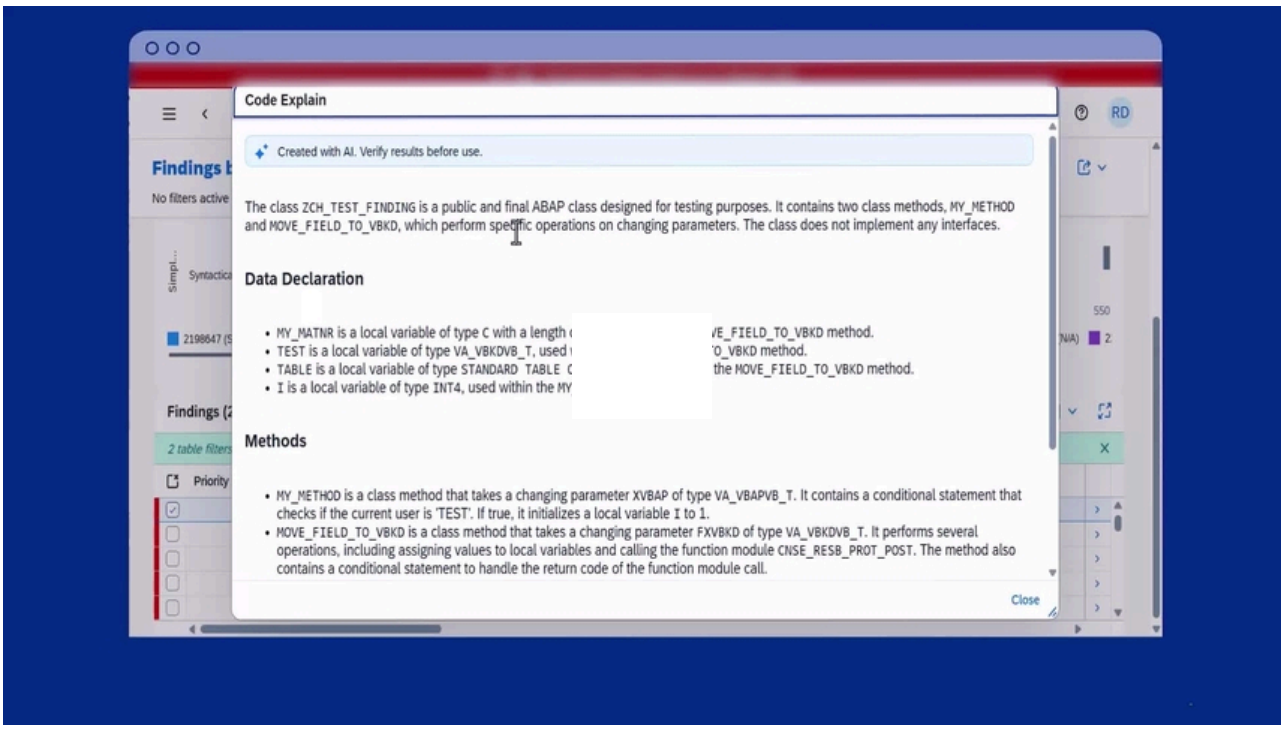
Open this video in a new window

Using ATC Explain for ATC Simplification Findings In-App



Open this video in a new window

Using Code Explain In-App



Open this video in a new window

Using the Custom Code Migration AI Assistant

Learn how to use the Custom Code Migration AI Assistant in Joule while developing in ABAP development tools for Eclipse.

Are you planning to convert your custom code from an SAP ECC system to an SAP S/4HANA system and need support on your transformation journey? The Custom Code Migration AI Assistant in Joule will be your starting point for understanding issues that

arise along the way. More specifically, it'll support you with your conversion by explaining ABAP Test Cockpit (ATC) findings to you in detail, including the corresponding simplification notes, and can, on top of that, access a large pool of simplification items that can affect custom code and answer your questions about these items. You can also have the Custom Code Migration AI Assistant give you explanations of ABAP programs.

Related Information

Opening the Custom Code Migration AI Assistant
Using the Explain ATC Findings Capability
Getting Custom Code Migration Assistance

Opening the Custom Code Migration AI Assistant

Learn how to open the Custom Code Migration AI Assistant in ABAP development tools for Eclipse.

Context

To be able to use the Custom Code Migration AI Assistant, you first need to learn how to open the Joule chat view. To do this, jump to Opening the Joule Chat View. From here, it depends on your use case how you activate the assistant:

Procedure

1. In the Joule chat window, select or type in **/docs** to start chatting about simplification items related to Custom Code Migration.
2. To chat with the assistant about SAP S/4HANA simplification items and Custom Code Migration, just start asking topic-related questions. You can find examples of this in topic Getting Custom Code Migration Assistance.
3. To get explanations of ABAP Test Cockpit (ATC) findings and to understand the issue described in the related simplification item, you'd open the Joule chat in a slightly different way. You can find more on that in Using the Explain ATC Findings Capability.
4. To get explanations of ABAP programs, right-click your program and choose **Explain**. The Joule chat window will then open and give you information about processing logic and business purpose of the programs as well as a technical explanation.

Related Information

Using the Explain ATC Findings Capability
Getting Custom Code Migration Assistance

Using the Explain ATC Findings Capability

Get detailed explanations of your ATC findings and relevant simplification items while developing in ABAP development tools for Eclipse by using the Explain capability.

This /explain ATC findings capability is specific to the Custom Code Migration use case. To get more general information on the /explain capability, see Using the Explain Capability.

About the Explain ATC Findings Capability

Joule in ABAP development tools for Eclipse offers you detailed explanations on ATC findings (simplification items only) resulting from ATC runs on ABAP development objects with a check variant for SAP S/4HANA Readiness performed in an SAP S/4HANA system.

You can use the Explain ATC findings capability on the following ABAP development objects:

- ABAP programs
- ABAP classes
- ABAP function modules

You can use the capability with the following ATC checks:

- S/4HANA: Field Length Extensions
- S/4HANA: Search for Database Operations
- S/4HANA: Search for Usages and Simplified Objects

After running the ATC, you'll get a list of findings that need to be addressed by fixing the source code. Each finding includes a link to a simplification item, which will be explained with the support of Joule. In the next section, you'll learn how to get to this point.

How to Get Explanations of ATC Findings

1. In your SAP S/4HANA system, right-click your development object and choose **Run As** > **ABAP Test Cockpit With...**.
2. In the dialog field, enter a check variant for SAP S/4HANA Readiness.
3. The ATC Problems view opens displaying your findings.
4. Right-click a finding and choose **Joule** > **Explain**.
5. In the Joule chat, you now have an explanation of the selected finding and a suggestion on how to fix the finding based on the related simplification note.

As you can see, this capability significantly reduces the effort needed to resolve ATC findings. In the next section, you'll learn how to get code proposals for ATC findings.

Getting Code Proposals

Learn how to get code proposals in Joule.

After you obtained ATC findings from SAP S/4HANA Readiness Checks, you can ask Joule for a code proposal. There are two ways in which you can do this:

Quick Link

1. After getting an explanation for your ATC findings, in the Joule chat window, you'll find a link at the very end of the explanation titled **Genrate Code Proposal**, given a code proposal is available for a particular finding.
2. Select this link to get a code proposal in the Joule chat window.
3. You now have three buttons to choose from at the top-right:

   By choosing **Apply** at the top-right of the code proposal, a new window opens where you can directly apply the proposal to your code, replacing the selected code. In the same window, you can use the buttons at the top-right of the code proposal to walk through the individual differences. Select **Finish** to apply the proposal.

   You can also choose **Insert** to insert the proposed code where your cursor is currently placed. With **Copy** you'll simply copy the proposed code.

1. In the ATC Problems view, double-click on a finding to navigate to the corresponding line of code.
2. Select the error marker next to the line of code. You'll now find the option **Generate Joule-based Code Proposal**.
3. Double-click on it and you'll get a code proposal in the Joule chat window. You can now proceed like you would with the option described above.

You're now familiar with the code proposals feature and can move on to the next topic, Getting Custom Code Migration Assistance, which goes hand in hand with this chapter and will support you further with your custom code migration journey and how to understand simplification items.

Getting Custom Code Migration Assistance

Get custom code migration assistance while developing in ABAP development tools for Eclipse by using the Custom Code Migration AI Assistant in Joule.

Context

You want to ask the Custom Code Migration AI Assistant questions about SAP S/4HANA simplification items and get all your answers in one place? Just open the Joule chat view and get started with your custom code migration project. You can find out, for example, what impact a specific simplification note has on your custom code. But there's more. You can get support from Joule from a large pool of simplification items that describe issues such as:

- Data model and data type changes
- Functionality no longer available
- Incompatible changes made to SAP development objects

Example Questions and Prompts

Please note that this list is not exhaustive. You can ask the assistant any question related to your custom code migration projects, such as:

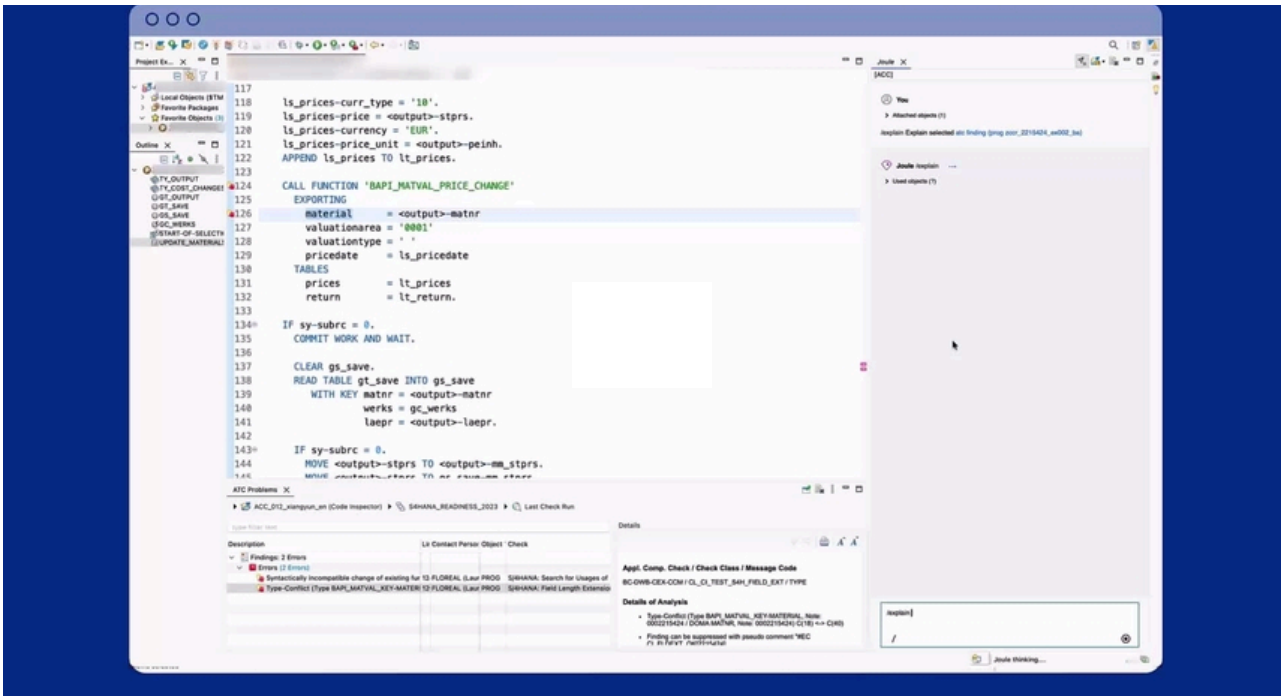
- What impact does SAP Note 2220005 related to Data Model Changes in Pricing and Condition Technique has on my custom code?
- In which table are the SAP SD pricing conditions entries stored in an SAP SAP S/4HANA system?
- How can a `SELECT` statement in the `KONV` table be transformed to the new SAP S/4HANA-compliant API?
- How does `MATNR` field length extension impact the code?

Video Tutorials for the Custom Code Migration AI Assistant

Learn about the two Explain use cases that are exclusive to Custom Code Migration.

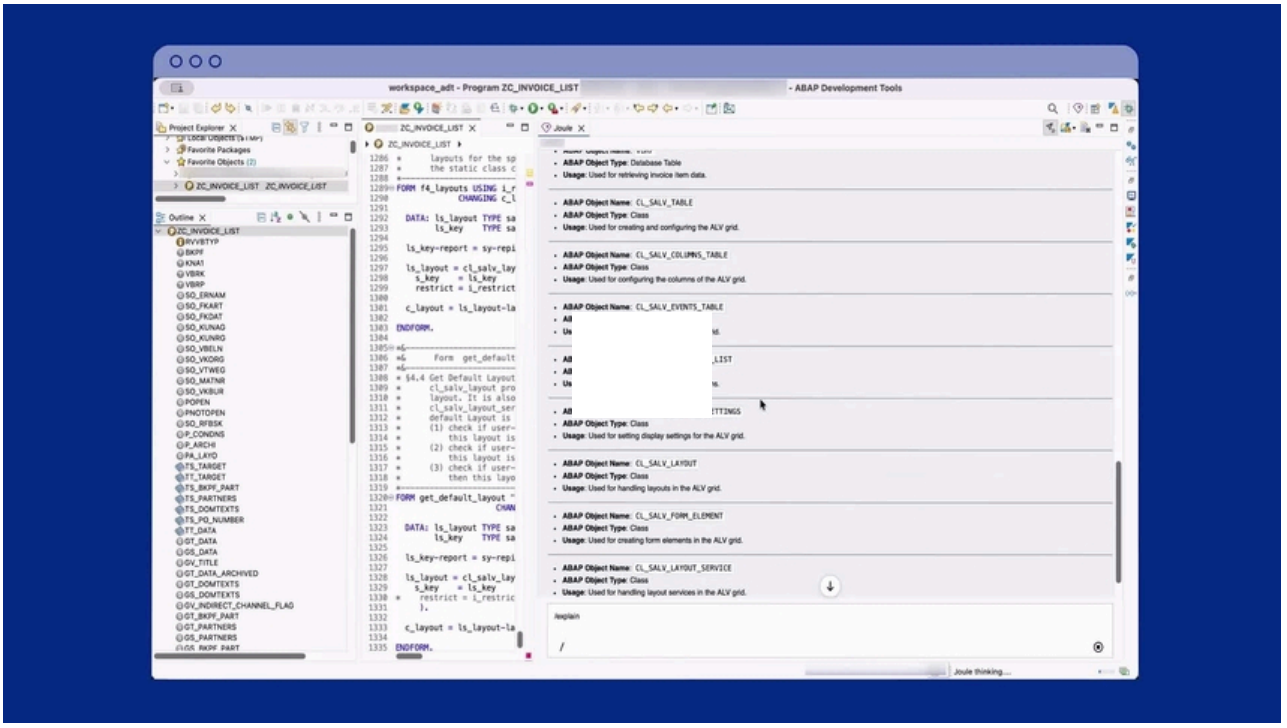
Video Tutorials

ATC Findings Explain for SAP S/4HANA Readiness Checks



Open this video in a new window

Program Explain



Open this video in a new window

Using Predictive Code Completion

Learn how to use the predictive code completion in ABAP development tools for Eclipse.

Context

The predictive code completion is enabled by default, but you can disable it manually if you prefer. When enabled, ghost text appears continuously during your coding process and is displayed in grey in the ABAP source code editor. If disabled, you can

manually trigger the generation of ghost text. As a developer, you decide how these code predictions enhance your work.

Procedure

1. Select **Toggle Automatic Triggering of Predictive Code Completion** () from the toolbar in ABAP development tools for Eclipse to enable or disable the predictive code completion. Alternatively, go to **Window** > **Preferences** > **Source Code Editors** > **Code Completion** and adjust the setting **Trigger predictive code completion (if supported by the backend system)**.

   → Remember

   The toggle appears in the toolbar only when a supported object type is opened and in focus in the source code editor. Refer to the List of Supported Object Types.

2. Begin typing in the source code editor and take a short break. If predictive code completion is enabled, ghost text appears in the ABAP source code editor at the point where you paused. You can then incorporate it into your code in the following way:

   - To accept the entire ghost text: Use the Tab key or click on the generated ghost text with your cursor.
   - To accept the next line: Use Ctrl + Down.
   - To accept the next word: Use Ctrl + Right.
   - To discard the ghost text: Use Esc.

   These shortcuts also appear in the **key bindings toolbar**, located above the generated ghost text in the source code editor. This toolbar is enabled by default. You can deactivate it in the preference settings. To do so, either select the three dots on the right-hand side of the key bindings toolbar or go to **Window** > **Preferences** > **Source Code Editors** > **Code Completion** and uncheck the setting **Show key bindings when ghost text is visible**.

3. **Optional:** Use the shortcut Alt + J on Windows or Option + J on Mac to manually trigger the predictive code completion. This works even if the toggle from the first step is disabled. To change the shortcut, go to **Window** > **Preferences** > **Keys** and search for `Trigger Predictive Code Completion`.

Related Information

Predictive Code Completion

Working with Predictive Code Completion

Get additional information on how to use the predictive code completion effectively, as well as its accessibility option.

Interaction of Predictive Code Completion and Code Completion Settings

Generally, the predictive code completion takes precedence over the code completion. If either the toggle in the toolbar or the preference page setting of the predictive code completion are switched on, the code completion is not triggered while writing code.

1. To understand the behavior of code completion and predictive code completion settings, go to the preference page by choosing **Window** > **Preferences** > **Source Code Editors** > **Code Completion**.
2. Note that the **Trigger code completion after typing '-', '~', '->', '=>', '\'** setting remains set to "on."
3. To check the effect of the **Trigger predictive code completion (if supported by the backend setting)** setting, ensure it is set to "on" in this preference page (or in the toolbar above your open source code editor).

When both settings are "on", the predictive code completion setting takes precedence.

Accessibility of Predictive Code Completion

The accessibility preference page now includes an option to display predictive code completion proposals as a dialog instead of ghost text, enhancing screenreader compatibility. This feature is useful for developers who rely on screenreaders for code accessibility and need a more inclusive coding environment.

To access this accessibility option go to Window > Preferences > ABAP Development > Accessibility > Predictive Code Completion and enable the setting Show proposals in a dialog box. This setting will disable the automatic triggering of the predictive code comletion. However, you can manually trigger the predictive code completion using the shortcut Alt + J on Windows or Option + J on Mac.

→ Tip

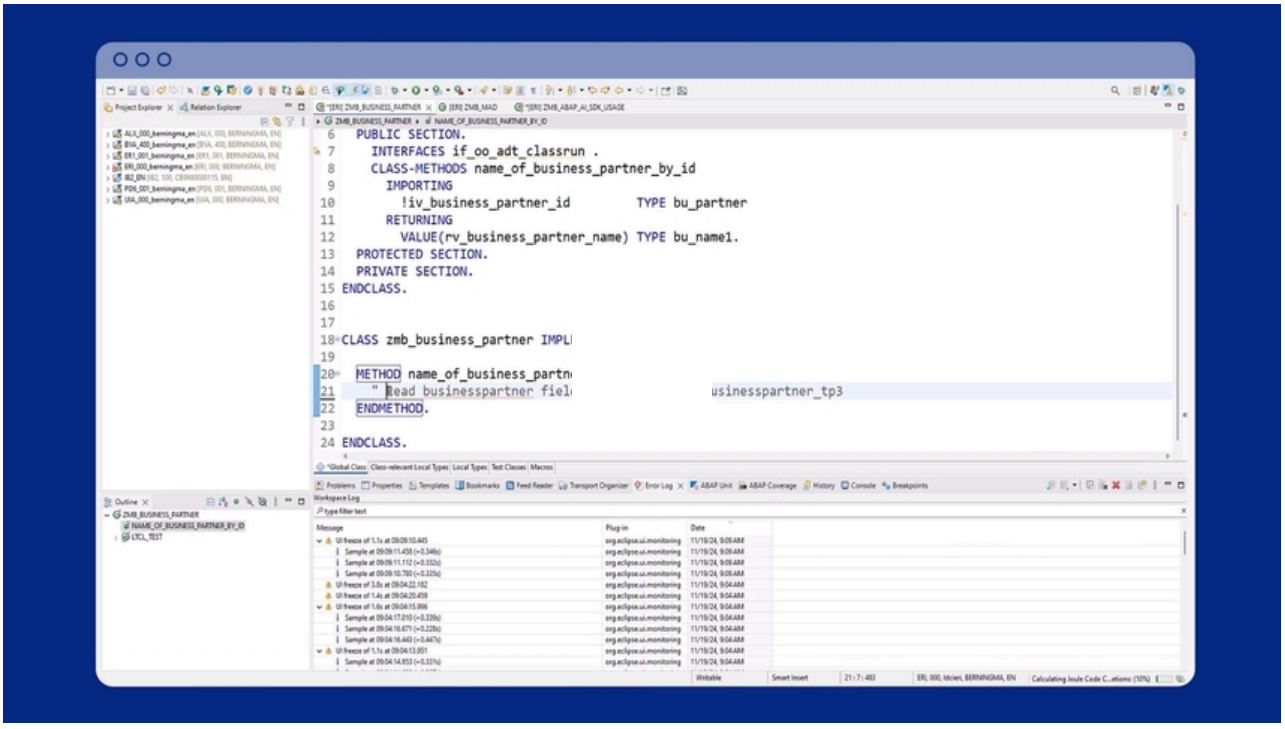
You can also toggle the triggering of predictive code completion using the search dialog (Ctrl + 3) and typing Toggle Automatic Triggering of Predictive Code Completion.

Video Tutorials for the Predictive Code Completion

Learn more about how to use the predictive code completion effectively and get an overview of its features with the following video tutorials.

Video Tutorials

Predictive Code Completion Based on Natural Language



Open this video in a new window

Using the ABAP Unit Test Generation Capability

Learn how AI-based ABAP Unit features can help you generate unit test methods and improve your existing test code.

Related Information

ABAP Unit Test Generation

Starting the ABAP Unit Chat

Use the ABAP Unit chat to perform various operations on your class to improve your test coding.

Context

To start the ABAP Unit chat within the ABAP development tools for Eclipse, follow the steps from one of the following locations:

- Project Explorer
- Outline view
- ABAP source code editor
- Relation Explorer
- ABAP Unit view
- ABAP Coverage view

Procedure

1. Select a class in one of the mentioned views or editors.
2. Open the context menu and select **Joule** > **Start Chat for ABAP Unit**.

   The **Joule** chat opens and asks what you want to do with your class.
3. **Optional:** If you want to add unit tests to validate your code, select **Generate Tests**. For more details, see Generating Tests with the ABAP Unit Chat.
4. **Optional:** If you want to specifically understand more details about your existing unit tests, like if there are test smells or which test patterns are used, choose **Explain <name of test class>**. For more information, see Test Code Explanations.
5. **Optional:** If you want to identify test-unfriendly dependencies to improve your test code, choose **Find Test-Unfriendly Dependencies**. For more information, see Dependency Analysis.
6. **Optional:** You can also use simple freestyle prompts by typing in the chat. See Using Freestyle User Prompts.

Related Information

ABAP Unit Test Generation

Test Generation

Generating Tests with the ABAP Unit Chat

The ABAP Unit chat enables you to generate ABAP Unit tests. It's useful for automating the creation of unit tests, ensuring code reliability and efficiency.

Prerequisites

Start the ABAP Unit chat for your class. See Starting the ABAP Unit Chat.

Procedure

1. After you've opened your class in the ABAP Unit chat, select **Generate Tests**.

   ### → Remember

   You can generate ABAP Unit tests for:

   - Public, protected, and private methods of global ABAP classes
   - Public methods of local classes within global class pools

2. You have the following options of generating test methods:

   - **Selected Methods...**: In the dialog that appears, select the methods for which you want to generate unit tests, and then select **OK**.
     - The **Show Metrics** dropdown menu allows you to display additional details by selecting:
       - Coverage: Selecting **Coverage** shows coverage results, such as branch coverage and statement coverage.
       - Usages: Selecting **Usages** shows the number of usages.
     - The **With Implementation** checkbox controls whether test generation uses the method implementation in addition to the method signature. By default, the checkbox is checked for private and protected methods and unchecked for public methods, and it remains read-only.
     - If your class doesn't contain any existing test classes, a response proposing a new test class is displayed.
   - **Most-Used Methods...**: Selecting **Most-Used Methods...** generates unit tests for the most-used methods of your class.
   - **Selected Local Class...**: In the dialog that appears, select the local class for which you want to generate unit tests, and then select **OK**.

     ### ⓘ Note

     The option **Selected Local Class...** is only visible if there are at least four local classes available.

3. Choose between **Add Tests to <name of existing test class>** and **Add Tests to New Test Class**.
4. **Optional:** When the generated code is displayed, select one of the buttons above the code:
   a. **Optional:** Select **Apply** to replace your current test code with the generated code. In the dialog, review the differences, then select **Finish**.
   b. **Optional:** Select **Insert** to insert the generated code at the current cursor position.
   c. **Optional:** Select **Copy** to copy the generated code to the clipboard.
5. **Optional:** You can select **Refactor Generated Code** to further improve the generated code. For more information, see Generated Test Code Refactoring.

Related Information

Test Generation

Using Freestyle User Prompts

Use freestyle prompts in the **Joule Chat** to interact with AI-powered features and quickly get answers, guidance, or assistance tailored to your needs.

Prerequisites

Start the ABAP Unit chat for your class before entering your prompts. See Starting the ABAP Unit Chat.

Procedure

- **Optional:** To generate unit tests for your class, follow these steps:
  1. Enter a freestyle prompt, such as: "`Generate unit tests.`"
  2. Select **Generate Tests**. For more details, see Generating Tests with the ABAP Unit Chat.
- **Optional:** To generate test doubles for database tables or CDS views in your class, follow these steps:
  1. Enter a prompt, such as: "`Generate test double for database table <name of database table>.`", or "`Generate test double for CDS view <name of CDS view>.`".
  2. Choose either **Generate Test Double for Database Table <name of database table>** or **Generate Test Double for CDS View <name of CDS view>**.

     > **Note**
     >
     > If the quick replies **Generate Test Double for Database Table <name of database table>** or **Generate Test Double for CDS View <name of CDS view>** aren't available, select **Send my prompt directly to AI**.

  3. Select **Generate in <name of existing test class>**.

     A response proposing a modified test class is displayed.
- **Optional:** To create test doubles for interface methods or function modules, follow these steps:
  1. Enter a prompt, such as: "`Generate test double for method <name of interface method> of interface <name of interface>.`", or "`Generate test double for function module <name of function module>.`"
  2. Choose either **Generate Responder for Interface Method** or **Generate Responder for Function Module**, based on your previous prompt.

     > **Note**
     >
     > If the quick replies **Generate Responder for Interface Method** or **Generate Responder for Function Module** aren't available, select **Send my prompt directly to AI**.

  3. Select **Generate in <name of existing test class>**.

     A response proposing a modified test class is displayed.

Related Information

Test Double Support
ABAP Unit Test Generation

Generating ABAP Unit Tests

Use SAP Joule for developers, ABAP AI capabilities to efficiently generate ABAP Unit tests for your classes.

Context

To generate ABAP Unit tests with ABAP development tools for Eclipse, follow the steps from one of the following locations:

- Project Explorer

- Outline view
- ABAP Coverage view
- Using quick replies in the ABAP Unit chat in **Joule**. See Starting the ABAP Unit Chat.

→ Remember

You can generate ABAP Unit tests for:

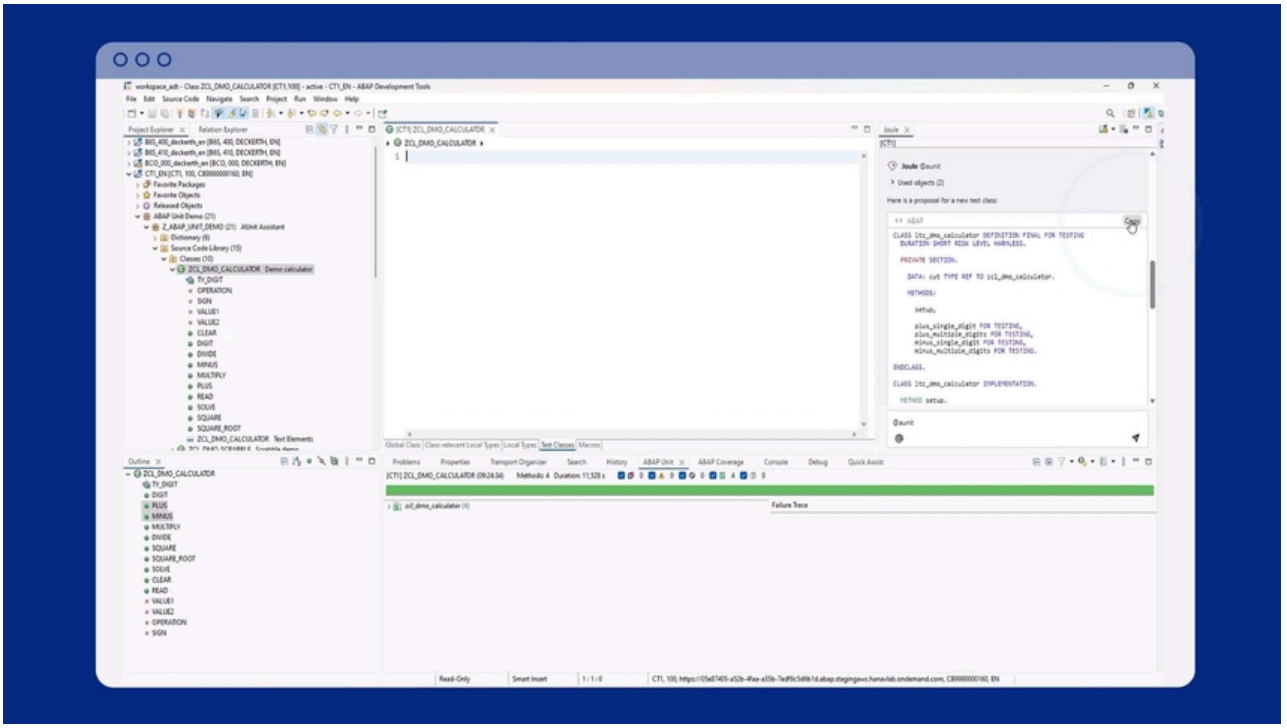
- Public, protected, and private methods of global ABAP classes
- Public methods of local classes within global class pools

Procedure

1. Navigate to one of the specified views and select a method from an ABAP class.
2. Open the context menu and select **Joule** > **Generate Unit Tests**.

   The ABAP Unit chat opens in **Joule**.
3. Choose between **Add Tests to <name of existing test class>** and **Add Tests to New Test Class**.
4. **Optional:** When the generated code is displayed, select one of the buttons above the code:
   a. **Optional:** Select **Apply** to replace your current test code with the generated code. In the dialog, review the differences, then select **Finish**.
   b. **Optional:** Select **Insert** to insert the generated code at the current cursor position.
   c. **Optional:** Select **Copy** to copy the generated code to the clipboard.
5. **Optional:** You can select **Refactor Generated Code** to further improve the generated code. For more information, see Generated Test Code Refactoring.

Example: Generating ABAP Unit Tests with Joule



Open this video in a new window

Related Information

Test Generation
ABAP Unit Test Generation

Analyzing Dependencies

Improve your understanding of your productive code by identifying test-unfriendly dependencies.

Context

With ABAP development tools for Eclipse, you can access the AI-based feature to find test-unfriendly dependencies from the following locations:

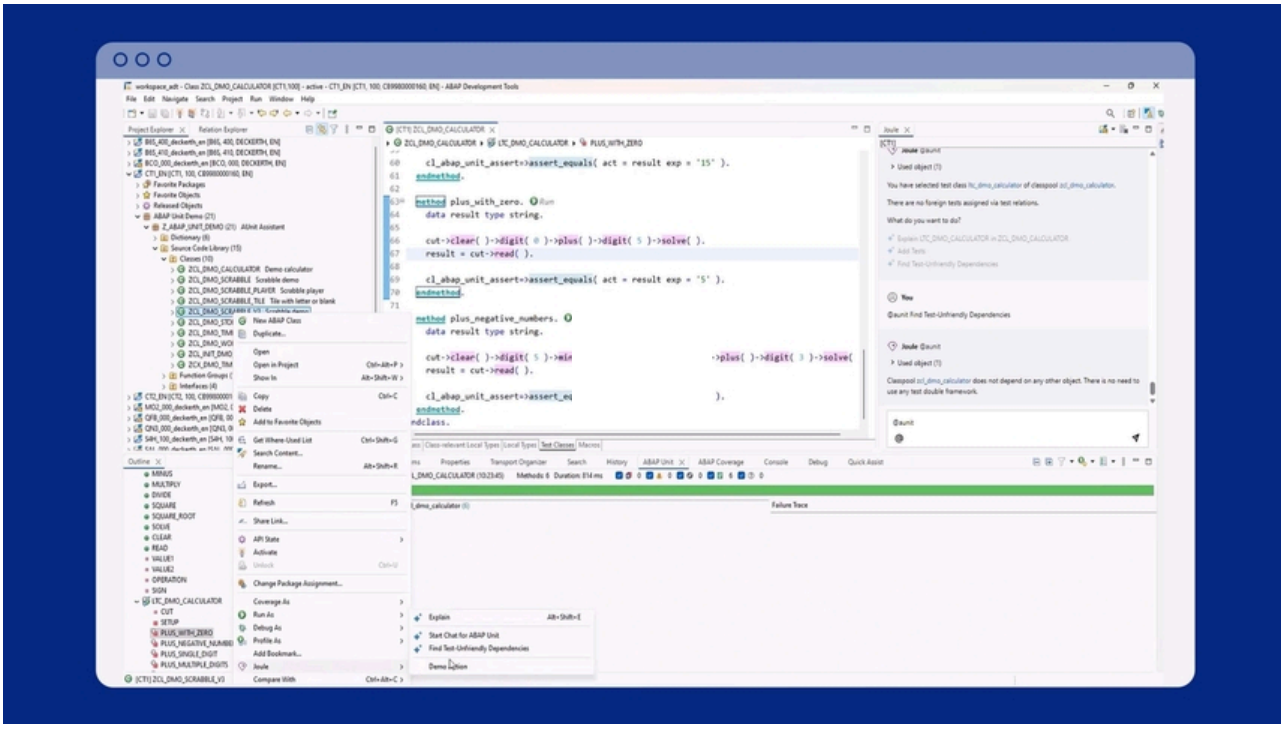
- Project Explorer
- Outline view
- ABAP Unit view
- ABAP Coverage view

Procedure

1. Select the class you're interested in from one of the listed views.
2. Open the context menu and select **Joule** > **Find Test-Unfriendly Dependencies**.

   If your class has dependencies on other objects, you will see links to those objects categorized under their respective object types. You can select these links to navigate directly to the objects.
3. To resolve the dependencies, you can configure test doubles. See Adding Test Doubles.

Example: How to Find Test-Unfriendly Dependencies



Open this video in a new window

Related Information

Dependency Analysis

ABAP Unit Test Generation

Adding Test Doubles

Using test doubles in your test code helps to replace dependencies to other development objects and increases the stability of your unit test results.

Context

The following procedure is only supported for these object types:

- Interfaces
- Function modules
- Database tables
- CDS views

Procedure

1. First you need to find out whether there are any test-unfriendly dependencies that need to be resolved. To achieve that, see Analyzing Dependencies.
2. If there are any test-unfriendly dependencies listed, you have the following three options:
    - You can navigate directly to the relevant development object in the source code editor and use quick assists with Ctrl + 1 to configure test doubles.
    - Based on the development object types used in your source code, you can use the following quick replies to configure test doubles:
        - **Isolate Interface Methods...**: Select **Isolate Interface Methods...** to open a dialog where you can choose an interface method for which a responder will be configured.
        - **Isolate Function Modules...**: Select **Isolate Function Modules...** to open a dialog where you can choose a function module for which a responder will be configured.
        - **Isolate Database Tables...**: Select **Isolate Database Tables...** to open a dialog where you can choose a database table to isolate.
        - **Isolate CDS Views...**: Select **Isolate CDS Views...** to open a dialog where you can choose a CDS view to isolate.
        - **Generate Responder for Function Module**: Select **Generate Responder for Function Module** to generate a test double for the used function module in a test class.
        - **Generate Test Double for Database Table <name of database table>**: Select **Generate Test Double for Database Table <name of database table>** to generate a test double for the used database table in a test class or helper class.
        - **Generate Test Double for CDS View <name of CDS view>**: Select **Generate Test Double for CDS View <name of CDS view>** to generate a test double for the used CDS view in a test class or helper class.

- You can also use freestyle user prompts to add test doubles. See Using Freestyle User Prompts.

Related Information

Test Double Support
ABAP Unit Test Generation

Getting Test Code Explanations

Use the AI-based Explain feature for ABAP Unit to gain deeper insights into your test code.

Procedure

1. Select the test class you're interested in from the **Project Explorer** or the **Outline** view.
2. Choose **Joule** > **Explain Test Class** from the context menu of the selected class.
3. **Optional:** Alternatively, you can start the ABAP Unit chat for the class you're interested in. See Starting the ABAP Unit Chat.
4. **Optional:** If the class pool contains a test class, you can choose **Explain <name of test class>**.

Related Information

Test Code Explanations
ABAP Unit Test Generation

Refactoring Generated Test Code

Use AI-based refactoring instructions to ensure that your generated test code meets the quality standards and the coding style you require.

Procedure

1. Start the ABAP Unit chat for the class you're interested in. See Starting the ABAP Unit Chat. Alternatively, you can also start by generating unit tests first, see Generating ABAP Unit Tests.
2. Select **Add Tests** to generate test code.
3. Select the public methods that you want to test in the **Outline** view or in the **ABAP Coverage** view.
4. Choose **Joule** > **Generate Unit Tests** from the context menu of the selected methods.
5. After adding the tests to your test class, select **Refactor Generated Test Code**.
6. Choose one of the listed instructions or type in your own instruction. For more information, see Generated Test Code Refactoring.

Related Information

ABAP Unit Test Generation

Splitting Test Classes

Reorganize your code by splitting test classes to enhance the structure and maintainability of your unit tests.

Context

When you use this AI-based feature, it splits your existing test class into two new test classes together with one abstract base class.

Procedure

1. Select the test class you're interested in from the **Project Explorer** or the **Outline** view.
2. Choose **Joule** > **Split Test Class** from the context menu of the selected class.

Related Information

Splitting Test Classes and Test Methods
ABAP Unit Test Generation

Splitting Test Methods

Use this AI-based feature to split existing test methods into multiple smaller ones, helping you restructure your code more effectively.

Context

When your test methods become too large or contain too many assertions, causing them to test multiple different things, consider splitting them into smaller, more focused test methods.

Procedure

1. Select the test method you're interested in from the **Project Explorer** or the **Outline** view.
2. Choose **Joule** > **Split Test Method** from the context menu of the selected test method.

Related Information

Splitting Test Classes and Test Methods
ABAP Unit Test Generation

Using Generators

You can use the following generators to simplify the steps involved in generating unit tests or creating repository objects for an OData UI service with RAP:

Using CDS Unit Test Class Generator

Use SAP Joule for developers, ABAP AI capabilities to boost your efficiency in generating unit test class for ABAP CDS entity.

Using the OData UI Service from Scratch Wizard with the Joule Chat

You can create all RAP-service related repository objects from scratch by using the OData UI Service from Scratch wizard. You don't need an already existing repository object, because you can start the wizard from an empty package. The wizard allows you to enter your business object requirements by using the Joule chat. For more information on RAP, see ABAP RESTful Application Programming Model.

Embedded Analytics Star Schema Generator Powered by AI

The CDS Analytical Model generator powered by AI is a tool used in ABAP analytics to create an analytical star schema from a CDS view entity, generating a cube definition. This tool is beneficial for efficiently structuring data models for analytical purposes, enhancing data querying and reporting capabilities.

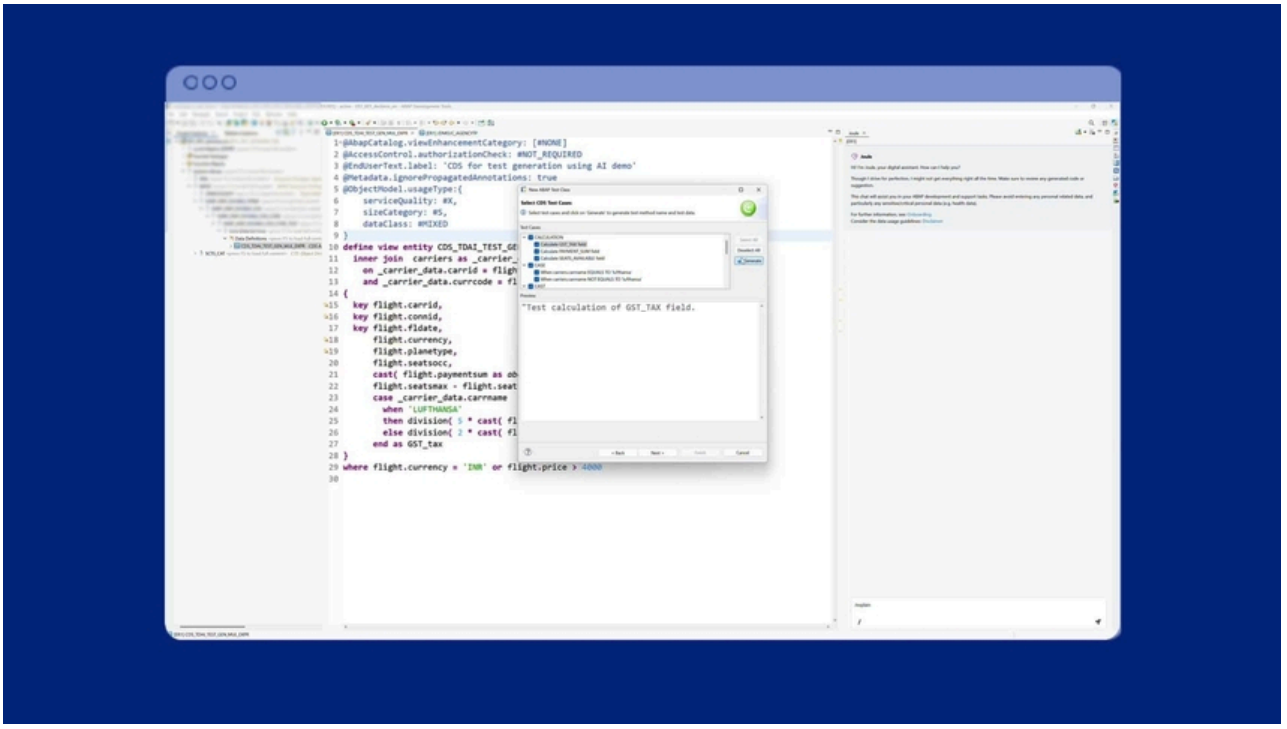
Using CDS Unit Test Class Generator

Use SAP Joule for developers, ABAP AI capabilities to boost your efficiency in generating unit test class for ABAP CDS entity.

Context

The AI assisted test class generates automated tests for ABAP CDS entity. This helps in generating more context-specific content in the ABAP Unit tests for a CDS entity. Hence, providing basic test coverage and improving developer efficiency.

Video Tutorial: Creating a New ABAP Test Class



Open this video in a new window

Procedure

1. In the **Project Explorer**, select the CDS entity.
2. Open the context menu and choose **Joule** > **New ABAP Test Class**.

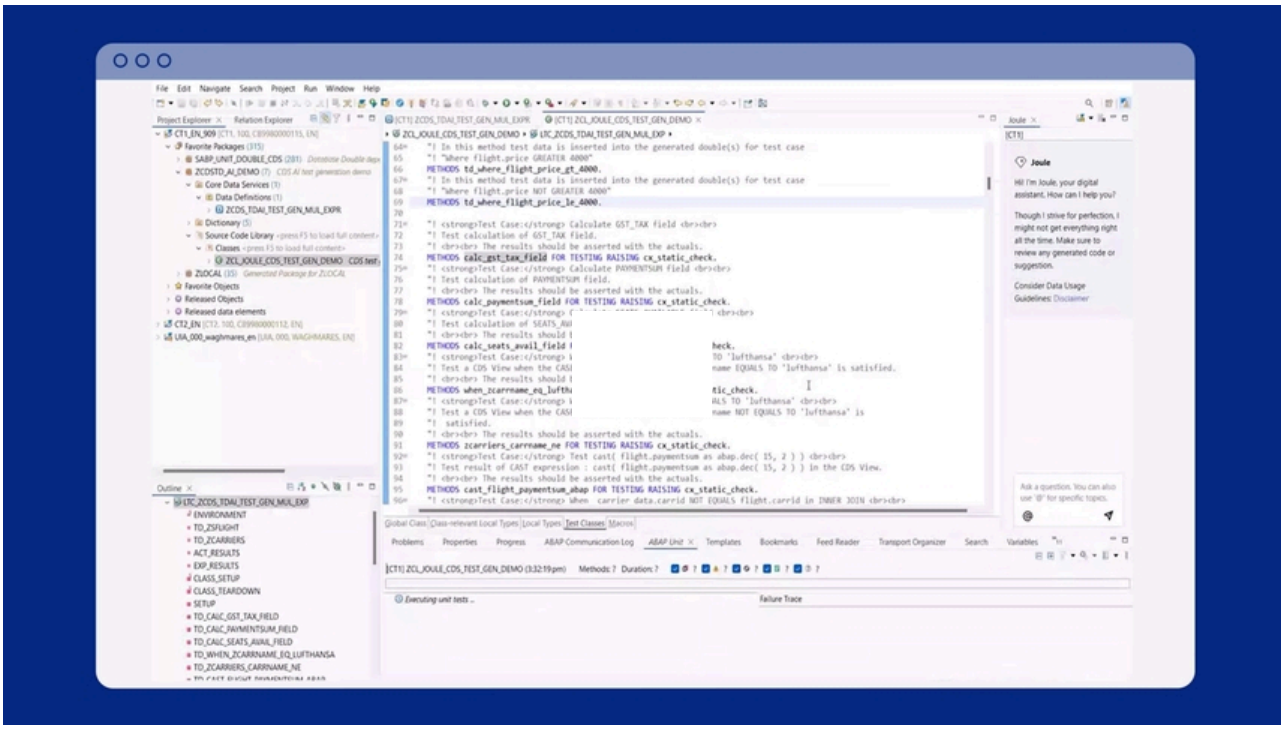
   This menu option is available only from the project explorer.
3. In the **New ABAP Test Class** wizard, enter the **Package**, **Name**, and **Description** for the ABAP test class to be created.
4. Choose **Next**.

   You can view the test case suggestions for all the semantics that are available in that CDS entity. The preview section shows the test case description.
5. Select the test cases and choose **Generate** to generate test method name and test data using AI.

   A preview of the test method source is generated for the selected test cases.
6. Choose **Next**.
7. Assign a transport request and choose **Finish**.

Example

Video Tutorial: Activating Test Class



Open this video in a new window

Using the OData UI Service from Scratch Wizard with the Joule Chat

You can create all RAP-service related repository objects from scratch by using the OData UI Service from Scratch wizard. You don't need an already existing repository object, because you can start the wizard from an empty package. The wizard allows you to enter your business object requirements by using the Joule chat. For more information on RAP, see ABAP RESTful Application Programming Model.

Prerequisites

This wizard allows you to start from an empty package without an already existing repository object in it. The only prerequisite for using this wizard is an already existing package.

ℹ Note

To leverage Joule's capabilities, you need to purchase an additional license: For SAP BTP ABAP environment and SAP S/4HANA Cloud Public Edition, see SAP Note 3571857.

Context

The **OData UI Service from Scratch** wizard allows you to create a RAP BO without the requirement of an already existing artifact and without having to manually create all the necessary repository objects yourself. You can start the **OData UI Service from Scratch** wizard from an empty package and the end result is a full-blown RAP UI service.

The following repository objects are created by the wizard:

- SAP object node type
- SAP object type
- Service binding
- Service definition
- Access control
- RAP projection behavior definition
- RAP behavior definition
- Data definition for CDS projection view entity
- Data definition for CDS view entity
- Metadata extension
- Database table for draft table
- Database table for persistent table
- ABAP class for ABAP behavior pool

Current Limitations of the OData UI Service from Scratch Wizard

The **OData UI Service from Scratch** wizard can help you creating a managed business object with internal early numbering. The wizard additionally offers the service types transactional with draft, transactional without draft and read-only. However, the wizard has the following limitations:

- Only one BO is created.
- Only UUID numbering is supported.

  ℹ Note

  - UUIDs will always be generated implicitly as key fields for you.
  - The UUID key fields will not be displayed in the configuration part of the generator.
  - You can't define specific fields as UUID **key fields** by yourself.

- No unmanaged BO is created.
- No Web APIs or analytical services are supported.
- Actions, determinations or validations aren't supported.
- No interface layers are created. You only have the base layer and the projection layer.
- Associations and cross-BO associations aren't supported.

Procedure

1. In the project explorer, open the context menu on your project or your package.
2. Choose **Generate ABAP Repository Objects.**
3. Select **ABAP RESTful Application Programming Model: OData UI Service from Scratch** as use case and choose **Next**.
4. If not already filled in automatically, enter your package name in the field **Package** and choose **Next**.

5. In the **Configure Generator** window, navigate through the wizard.

    a. Under the tab **Service Configuration**, you can select a service type (read-only, with draft, or without draft), as well as a prefix and/or a suffix with a maximum length of three characters for the artifacts that are going to be generated for you. If you start the wizard from an already existing package with a specific namespace, the wizard automatically assigns the same namespace of your package to the generated artifacts.

    > → **Recommendation**
    >
    > We recommend that you use at least a suffix.

    b. Additionally, you need to provide a project name. If you use the Joule chat for your BO creation, the project name will be generated for you.

6. In the **Configure Generator** window, you can then proceed by using the Joule chat to define the requirements for your business object.

    a. In the Joule chat window on the right-hand side of the wizard, enter a prompt with the requirements for your BO and press **Enter**. For instance, you can use the following example as a prompt: "Create a Travel BO with two entities".

    b. You have the option to use an already existing template, such as a database table or a DDIC structure to create an entity. You must specify in the prompt which template you want to use.

    c. Based on your prompt, Joule now proposes you a list of BO entities and BO entity fields. If this suggestion meets your requirements, choose **Accept**. The wizard now fills the configuration fields automatically.

    d. You can still use the Joule chat again for further adjustments or you can adjust the fields manually. For more information on how to do so without the Joule chat, see Using the OData UI Service from Scratch Wizard.

    e. Choose **Next** in order to get your generated artifacts.

7. The wizard shows you a list of the repository objects that are going to be generated and shows you a preview for most of the artifacts. If the list meets your requirements, choose **Next**.

8. Select a transport request and choose **Finish**.

Example: OData UI Service from Scratch Wizard with Joule Chat

This topic describes how you can create an OData UI service without an already existing repository object by using the **OData UI Service from Scratch** wizard with the aid of the Joule chat.

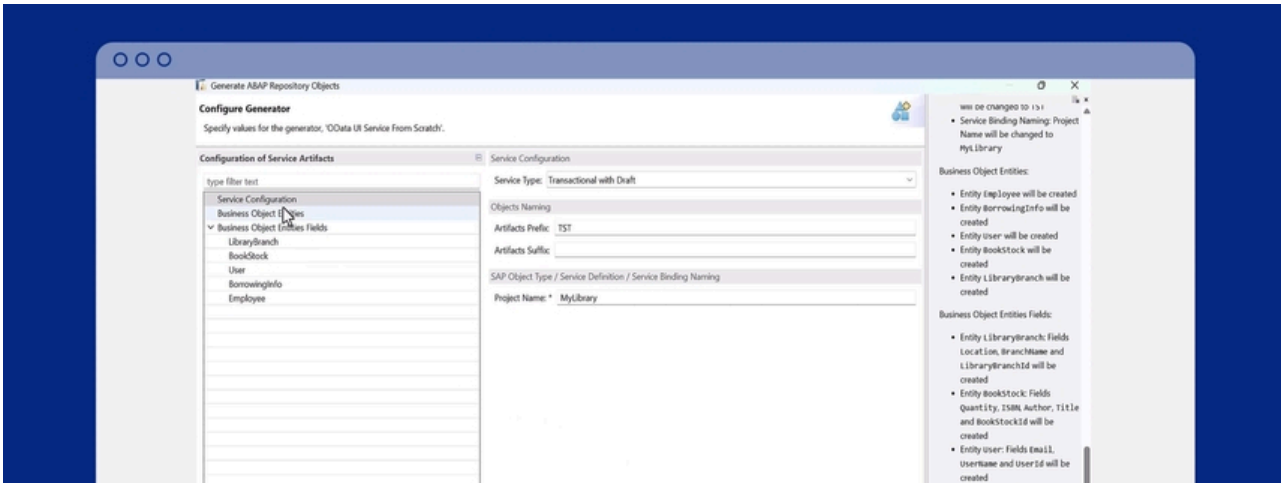
Generation Result of the OData UI Service from Scratch Wizard

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Example: OData UI Service from Scratch Wizard with Joule Chat

This topic describes how you can create an OData UI service without an already existing repository object by using the **OData UI Service from Scratch** wizard with the aid of the Joule chat.

Demo Video of the OData UI Service from Scratch Wizard





Open this video in a new window

Below you have a step-by-step description of the **OData UI Service from Scratch** wizard using another example.

Prepare a Package

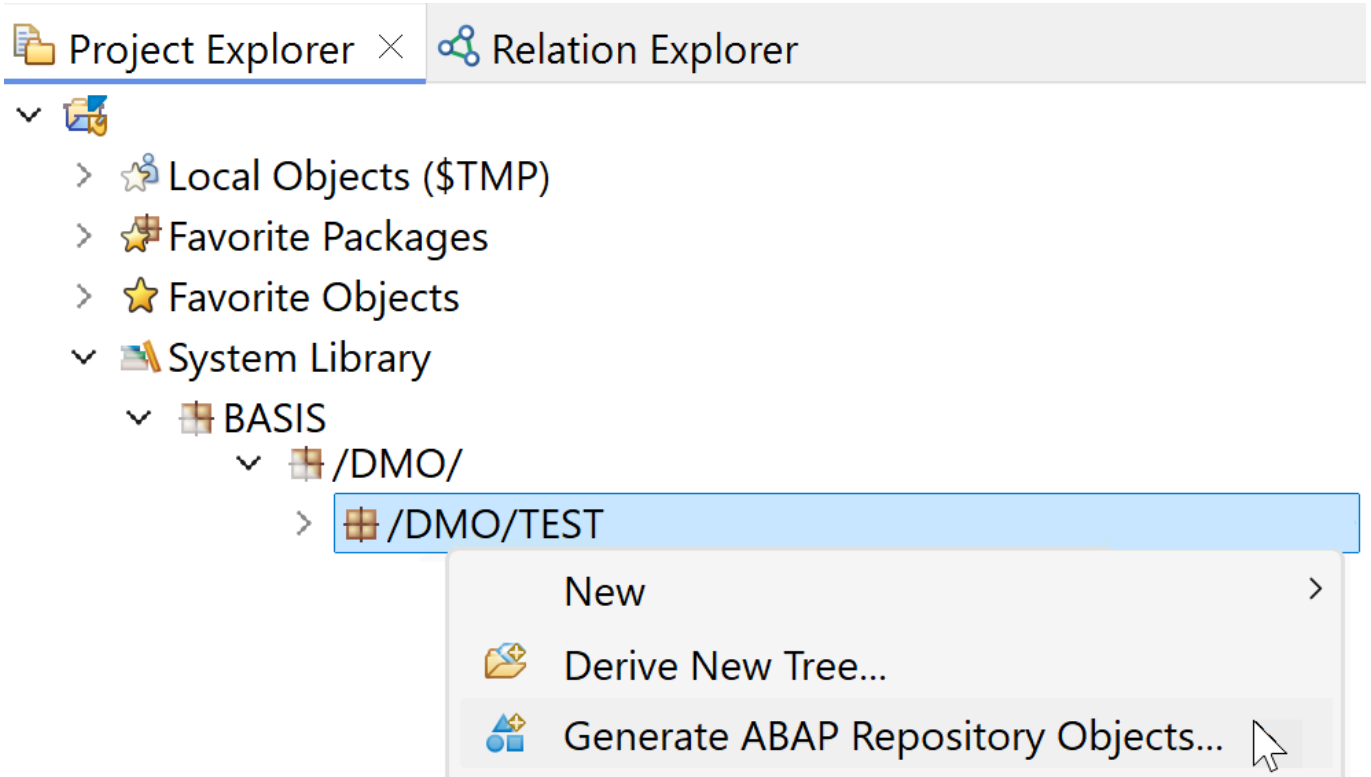
There are no prerequisites for starting the wizard other than having already created a package. You can then start the wizard based on your package. If you start the wizard from your project, you need to enter your package name in the field **Package** as described in Using the OData UI Service from Scratch Wizard.

Note

To leverage Joule's capabilities, you need to purchase an additional license: For SAP BTP ABAP environment and SAP S/4HANA Cloud Public Edition, see SAP Note 3571857.

Use the OData UI Service from Scratch Wizard

1. In the project explorer, open the context menu on your package and choose **Generate ABAP Repository Objects**. In this example, a package with the namespace `/DMO/` was used:



Note

You can also open the context menu on your project, but then you have to enter your package name in the field **Package**.

2. Choose **ABAP RESTful Application Programming Model: OData UI Service from Scratch** as use case.

3. The **Configure Generator** window opens. Additionally, the Joule chat appears and allows you to enter a prompt based on the requirements for your business object. For this example, the following prompt is going to be used: "Create a Travel BO with two entities". Additionally, the suffix `_G` was added.

4. After the prompt was sent, Joule proposes you a list of BO entities and BO entity fields. If this suggestion meets your requirements, choose **Accept**.

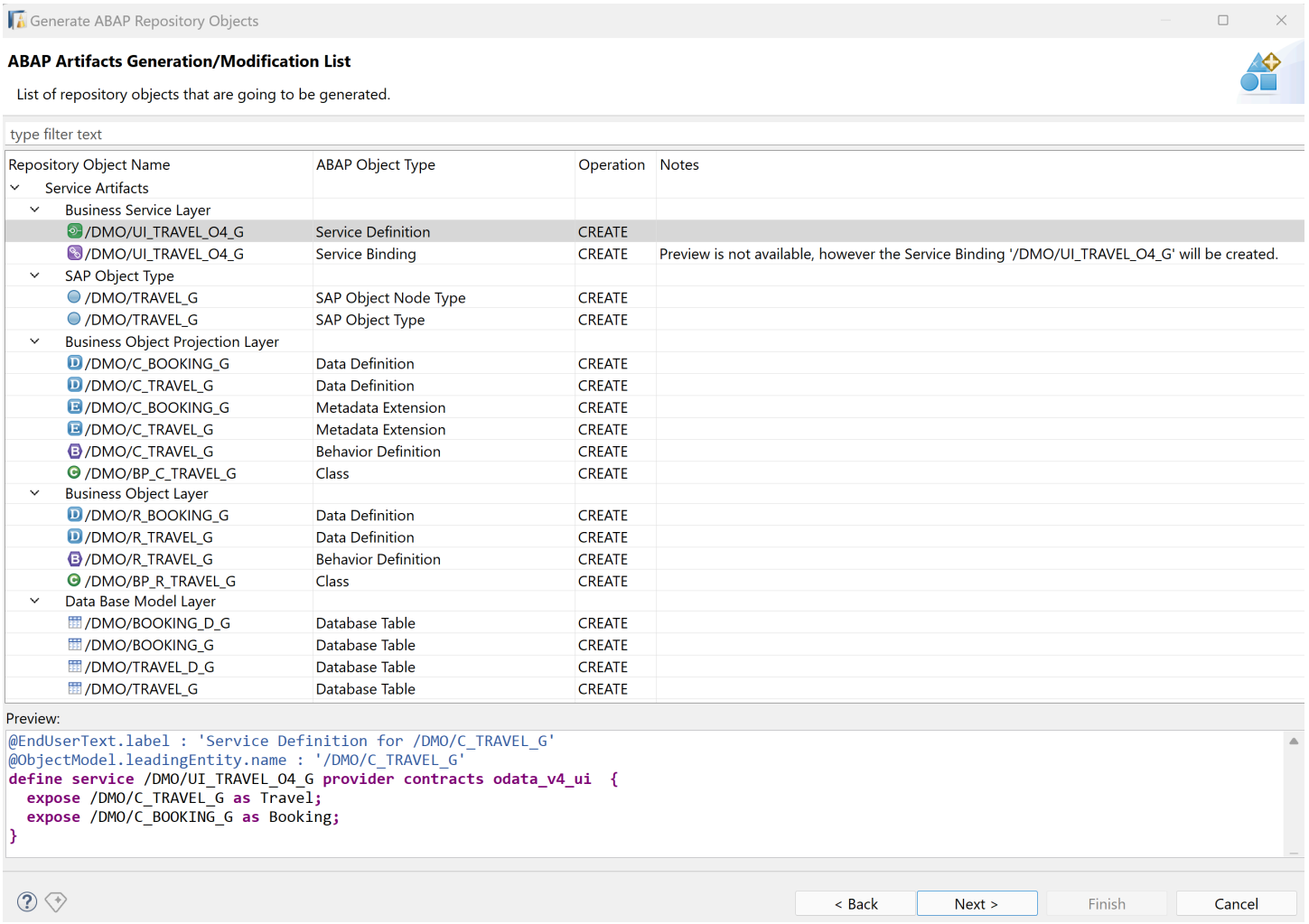
5. The wizard fills the configuration fields automatically. You can still navigate through the wizard and edit the business object entity names and the business object entity field names provided by Joule. Decide on a service type. For more information concerning the configuration of the generator, see Using the OData UI Service from Scratch Wizard.

6. Check if everything meets your requirements. Then choose **Next**.

7. Select a transport request and then choose **Finish** to start the generation of all repository objects.

Generated Repository Objects

The **OData UI Service from Scratch** wizard generates the following repository objects:



Since the wizard started from a package with the namespace `/DMO/`, all repository objects start with exactly this namespace.

You can easily add more BO entities to the composition structure or add other features to the generated objects. These features can be related to the data model, for example value helps or search capabilities, or related to the behavior. In the behavior definition, you can add business logic by defining and implementing actions, validations, or determinations.

ℹ Note

When adding new CDS view entities to the compositional structure of the data model, make sure that you also extend the behavior definition for the new BO entities. Otherwise your business service will show errors.

Related Information

Generation Result of the OData UI Service from Scratch Wizard

Generation Result of the OData UI Service from Scratch Wizard

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Result

As a result, the **OData UI Service from Scratch** wizard generates all the repository objects required for your BO. All objects are activated automatically. Based on your service type selection in the wizard, the RAP business object is either read-only, draft-enabled or not draft-enabled. It is also exposed as a UI service. By opening the service binding editor, you can publish the service locally and open the **Fiori Elements App Preview** to see the result on the UI. For more information, see Previewing the Resulting UI Service.

You can use all repository objects as usual and modify them. You can add data model options, for example value help or search capabilities to CDS view entities. The business object is also enabled for development extensibility. You can extend the behavior with actions and functions or other business logic with validations and determinations. For more information, see Validations, Determinations or Actions. In general, the generated business object is editable and useable in the same way as a manually developed business object.

Data Model

The generation wizard creates the business object from scratch. The data model in the CDS view entities uses all fields that are defined in the database table. It adds CDS annotations that can be retrieved from the database modeling, for example `@Semantics` annotations for administrative fields, which are needed for a managed BO. The wizard also creates alias names for the elements in the CDS view entity that represents the BO entity. The alias names are created in camel case.

Additionally, the wizard generates the syntax for compositions for a BO with multiple entities. The cardinalities to the child entities in the compositions are based on what you have entered in the wizard before. For more information about compositions, see Providing CDS Data Model with Business Object Structure.

The CDS projection view entity contains all relevant fields for the transactional data model and the dedicated admin field for ETag handling.

Metadata Extension

For UI scenarios, the generation wizard creates a metadata extension object with the basic UI-related annotations for a UI service. The UI service that is generated creates a basic list report application that exposes all transaction-enabled fields on the UI. The fields are annotated with the relevant `@UI.lineItem` and `@UI.identification` annotation.

If you want to change the UI, you can do so by using UI annotations. You can use the SAP Fiori Feature Showcase App for RAP to get an overview and examples for potential UI features.

Behavior

The generation wizard creates a managed RAP BO. The behavior definition defines strict mode. It contains the most basic behavior characteristics, for example the lock master and the authorization master for global authorization control. In addition, the ETag master is defined.

The generated RAP BO defines the standard operations and the draft actions (if it is draft-enabled). In addition, the administrative fields are set to read-only and the field mapping is defined.

The RAP BO uses managed internal early numbering. For more information, see Internal Early Numbering.

Business Service

The service definition exposes the generated CDS projection view and the service binding exposes an OData V4 UI service.

If you want to deploy your app to a SAP Fiori launchpad and make it accessible for business users, see App Deployment.

Embedded Analytics Star Schema Generator Powered by AI

The CDS Analytical Model generator powered by AI is a tool used in ABAP analytics to create an analytical star schema from a CDS view entity, generating a cube definition. This tool is beneficial for efficiently structuring data models for analytical purposes, enhancing data querying and reporting capabilities.

Prerequisites

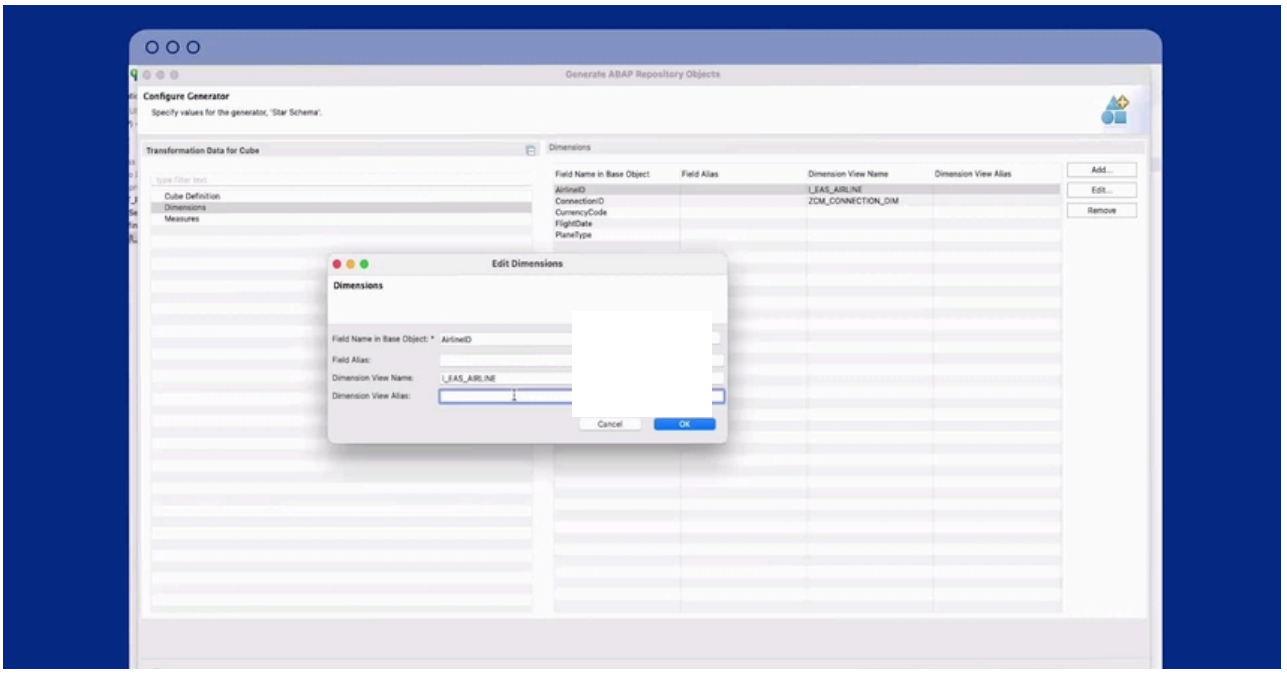
Make sure you have an active CDS view entity to be able to start the generator.

Context

The star schema is a structure that is often used in data modeling in ABAP analytics. Using the CDS CDS Analytical Model generator powered by AI in ABAP development tools for Eclipse allows you to seamlessly generate an analytical star schema for your data model.

Based on the root CDS view entity, the following elements are generated by default:

- **Cube definition:** the center of a the analytical data model, the cube view is a CDS view entity annotated with the annotation `@Analytics.dataCategory: #CUBE`. A cube consists of dimension fields with associations to dimension views and measures.
- **Dimension fields** (like Product, Region, Time, Customer) that categorize and provide context.
- **Measure fields:** transactional data which is measurable.





Open this video in a new window

Procedure

1. In the project explorer, open the context menu on your CDS view entity.

   > **Note**
   > You can also open the context menu on your package or project, but then you must enter the name of your CDS view entity in the field **Referenced Object**.

2. Select **Star Schema** under **ABAP Embedded Analytics** and choose **Next**.
3. Enter the package name in the field **Package**. If it's filled automatically, check that the correct package is selected. Choose **Next**.
4. In the **Configure Generator** window, navigate through the generator and adjust any elements as needed.
5. Under **ABAP Artifacts Generation/Modification List**, the artifacts to be generated are listed. Check if all necessary artifacts are listed, including:

   a. Cube definition

   b. Dimension fields: these can then be associated to a suitable dimension view. The association on condition will be generated implicitly, depending on the key fields of the dimension view and the corresponding dimension field in the cube view.

   c. Measures: the developer can select the necessary aggregation strategy for each measure field.

   If all required artifacts are listed and filled with the elements and properties to be generated, choose **Next**.

6. Select a transport request, if necessary, and choose **Finish**. Your hierarchy will now be generated.
7. You will now be asked if you want to open your generated hierarchy artifacts in the editor. You can choose to open and edit any of the generated artifacts by selecting **Open**.
8. You're now finished. You can find the generated CDS view entities under the **Data Definitions** section under **Core Data Services** section in your package in ABAP development tools for Eclipse.

Related Information

Star Schema (ABAP Analytics Guide)

Using RAP Business Logic Prediction

Generate the implementation of a RAP determination or validation with the aid of the AI-based **RAP business logic prediction**.

Prerequisites

Navigate to a behavior definition that has a determination or validation. Then navigate to an implementing class of the behavior definition and into the method implementation of a determination or validation. The method must be empty for the **RAP business logic prediction** to function. However, the capability still functions if the method contains only comments.

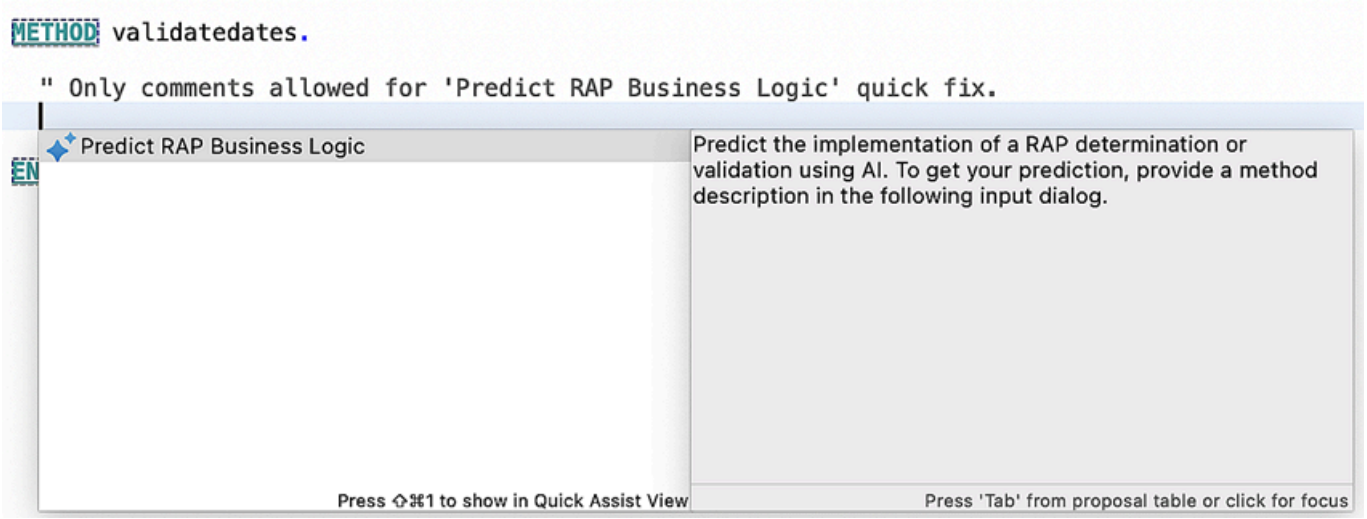
Context

When you use the **RAP business logic prediction** in ABAP development tools for Eclipse, the generated method implementation appears in the source code editor within the method implementation of your determination or validation. Follow this step-by-step

procedure to access the RAP business logic prediction.

Procedure

1. To open the RAP business logic prediction, select the quick fix in the empty determination or validation method implementation using Ctrl + 1 or via context menu.



Predict RAP Business Logic in Source Code Editor

→ Tip

The quick assist view also shows which quick fixes are suitable in a specific development context. You can open the quick assist view choosing Window > Show View > Other > Quick Assist.

2. Select the quick fix Predict RAP Business Logic to proceed.
3. The Predict RAP Business Logic dialog opens. To get a prediction, provide a method description in the input dialog. The method description can be similar to a prompt in the Joule chat.

   For a determination or validation the description can contain the following information:

   - What kind of data needs to be read? Provide read entity information.
   - What is to be determined or validated and how?
   - What fields need to be considered?
   - Are there any error scenarios or error messages?
   - What about edge cases?

4. Choose Run to review the generated method implementation in the source code editor.

→ Tip

If you like to review the code changes in the code review dialog, adjust the preference settings for the quick assist. Choose Window > Preferences > ABAP Development > Editors > Source Code Editors > Quick Assist and tick the box Preview changes (if supported by selected quick assist).

Related Information

RAP Business Logic Prediction
Determinations
Validations
Developing Determinations
Developing Validations

Using the Consume Capability

Generate an OData request and ABAP code for consuming a corresponding OData call via the OData Client Proxy with the aid of Joule.

Prerequisites

To use the **Consume** capability, you need a service consumption model with a **model class**. This model can be created in ABAP development tools for Eclipse by providing the metadata of the service you want to consume.

For further information, see Creating Service Consumption Model.

Context

The **Consume** capability can be accessed through the slash command **/consume** and needs to be distinguished from the standard Joule chat view. Use this step procedure to get started:

Procedure

1. Open the service consumption model of the OData service that you want to consume.
2. Start the Joule chat with either of the following two options:
    a. Choose **Joule** > **Consume** from the context menu by selecting an object in the project explorer.
    b. Open the Joule chat view and select the slash command **/consume** in the input field at the bottom of the view.

Results

Following these steps, Joule chat provides a few example prompts and further information to continue generating an OData request and ABAP code for your use case.

Related Information

Working with the Consume Capability
Creating Service Consumption Model

Working with the Consume Capability

Learn how to use the **Consume** capability productively and effectively.

Once you have selected the **Consume** capability in the Joule chat view, you have two options:

1. You already know the OData request you want to consume? In this case, simply enter the HTTP request in the Joule chat (for example, `GET/Campaigns?$select=ID`). The Joule chat then generates the corresponding ABAP code to consume the provided OData request.
2. You need an OData request that solves a certain task and the corresponding ABAP code to consume it? In this case, enter a prompt in the Joule chat, describing your OData request in natural language (for example, "I want to read the field ID from all campaigns.") The Joule chat generates an OData request and corresponding ABAP code.

At this point, you can either implement the provided coding or adjust the generated OData request if needed and re-enter it in the Joule chat.

Note

Verify that the OData request is generated for the correct entity by **/consume**. If the entity isn't correct, prompt Joule with the correct entity in your ongoing chat with **/consume**. The same approach applies to wrongly selected actions or functions.

Tips & Tricks

- For OData requests that contain a payload, provide the payload after the request in a new line.

  ### Example

  Here's an example for creating an **Employee**:

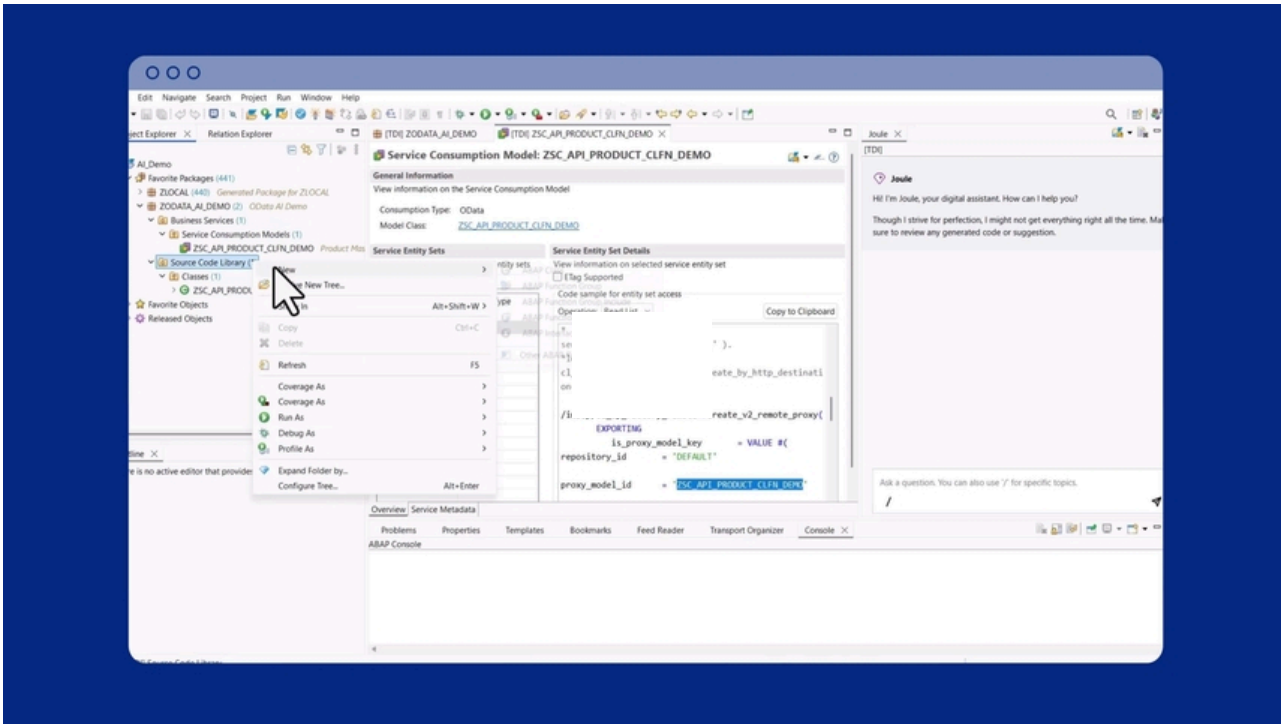
  ```
  POST /Employees

  {
    "ID": "1",
    "createdAt": "2023-08-21T15:00:00Z",
    "mail": "employee_1@example.com",
    "name": "First Employee"
  }
  ```

- If you need further details of the structure of the HTTP syntax, simply use **/consume** and ask "Explain the HTTP request syntax."

Video Tutorial

Work with the Consume Capability



Open this video in a new window

Summing Up

When the result fits your task, you can copy the provided ABAP code and implement it as a local class where needed. For further information on how to use the **OData Client Proxy**, you can refer to the Client Proxy User Guide OData Client Proxy User Guide.

Related Information

Consume
Using the Consume Capability